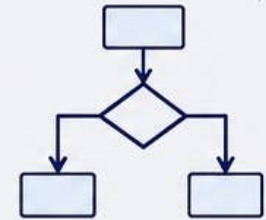


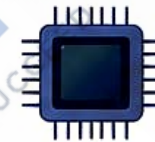
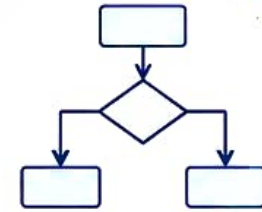
1010101010
0101010101
1010101010
0101010101



Notes



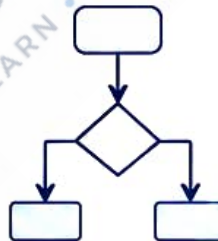
1010101010
0101010101
1010101010
0101010101



Unit-I



10110
01001
10101



Introduction to Artificial Intelligence

Unit - I
(112225)

1. What is Artificial Intelligence ?

- Artificial Intelligence (AI) is the branch of computer science that deals with creating machines or systems that can perform tasks which normally require human intelligence.
- It enables computers to learn, reason, solve problems, understand language and make decisions.

"AI is the ability of a machine to mimic human intelligence."

2. Why AI is Important ?

- Automates repetitive and complex tasks.
- Improves accuracy and efficiency.
- Helps in decision making.
- Solves problems that are difficult for humans.
- Used in many real-world applications (e.g., healthcare, education, finance, robotics).

3. History / Evolution of AI

- 1950s** - Term "Artificial Intelligence" coined by John McCarthy.
- 1960s** - Early AI programs and symbolic reasoning (e.g., Logic Theorist).
- 1970s** - AI Winter (lack of progress & funding).
- 1980s** - Expert systems and knowledge-based approaches.
- 1990s** - Machine learning and data-driven methods.
- 2000s** - Big data, cloud computing, deep learning.
- 2010s+** - Modern AI (deep learning, NLP, computer vision, generative AI).

4. Goals of AI

- Create intelligent machines.
- Make machines learn from experience.
- Develop systems that can reason, plan and solve problems.
- Achieve human-like performance in specific tasks.

5. Types of AI

Narrow AI (Weak AI)	• Designed for specific tasks. • Examples: Siri, Alexa, image recognition.
General AI (Strong AI)	• Human-like intelligence (not yet achieved). • Can perform any intellectual task.
Super AI	• Far beyond human intelligence (future concept). • Can improve itself.

6. Key Characteristics of AI

- **Learning** - Improves with experience.
- **Reasoning** - Draws conclusions from data.
- **Problem Solving** - Finds solutions to complex problems.
- **Knowledge Representation** - Stores and uses information.
- **Natural Language Processing** - Understands human language.
- **Perception** - Interprets data from environment (e.g., images, audio).
- **Planning** - Decides best course of action.

7. AI vs Human Intelligence

AI	Human Intelligence
• Works on data and algorithms.	• Works on experience, emotions and intuition.
• No emotions.	• Has emotions and feelings.
• Fast and accurate in calculations.	• Can be creative but sometimes makes mistakes.
• Limited to programmed tasks.	• Can learn, adapt and think beyond limits.

8. Real-World Applications

- **Healthcare** - disease diagnosis, robot surgery.
- **Education** - personalized learning, virtual tutors.
- **Finance** - fraud detection, risk analysis.
- **Transportation** - self-driving cars, traffic control.
- **Robotics** - industrial automation, home robots.
- **Language** - translation, chatbots, voice assistants.
- **Entertainment** - recommendation systems, gaming.

9. Challenges in AI

- Lack of human-like reasoning and creativity.
- **Bias** in data and algorithms.
- High computational and storage requirements.
- Ethical and privacy concerns.
- Job displacement.
- Need for large amount of data.

AI is not just about machines,
it's about creating a smarter future.

AI Problems and Techniques

Unit - II
(112225)

1. Problem Characteristics

AI problems are usually defined by the following characteristics:

- Well-defined or ill-defined.
- Deterministic or non-deterministic.
- Static or dynamic.
- Fully observable or partially observable.
- Discrete or continuous.
- Single agent or multi-agent.
- Known or unknown environment.
- Simple or complex.

Types of Problems

Type	Description
Well-defined	Clear goal, clear rules (e.g., 8-puzzle).
Ill-defined	Goals are vague (e.g., natural language understanding).
Deterministic	Next state is known (e.g., chess).
Non-deterministic	Multiple possible outcomes (e.g., robot in uncertain environment).
Static	Environment does not change (e.g., puzzle).
Dynamic	Environment changes over time (e.g., real-world driving).
Fully observable	Complete information available (e.g., chess).
Partially observable	Incomplete information (e.g., game playing with hidden state).
Single agent	One agent works (e.g., 8-puzzle).
Multi-agent	Multiple agents interact (e.g., game theory).
Discrete	Finite set of states/actions (e.g., board games).
Continuous	Infinite set of values (e.g., robot navigation).

2. Problem Formulation

Problem formulation means converting a real-world situation into a well-defined problem with:

- Initial state (starting point)
- Goal state (desired solution)
- Actions (possible moves)
- Transition model (result of actions)
- Cost function (measure of performance)

Example (8-Puzzle):

Initial State :

2	8	3
1	6	4
7	0	5

 Goal State :

1	2	3
8	0	4
7	6	5

Actions : Move tile (Up, Down, Left, Right)
Cost : 1 per move

3. Search Algorithms

Search algorithms are used to find a sequence of actions that leads from the initial state to the goal state.

Types of Search Techniques

Technique	Key Idea / Features
Breadth-First Search (BFS)	• Explores level by level. • Complete (if branching factor is finite). • Not optimal (unless all step costs are equal).
Depth-First Search (DFS)	• Explores deepest first. • Uses less memory. • Not complete (may get stuck).
Hill Climbing	• Uses local information. • May get stuck in local maxima.
Best First Search	• Uses heuristic function ($h(n)$). • Greedy approach.
A* Algorithm	• Uses $f(n) = g(n) + h(n)$. • Complete and optimal (with admissible h).
Constraint Satisfaction	• Finds solutions that satisfy all constraints. • (Used in scheduling, coloring, etc.)

4. Constraint Satisfaction Problem (CSP)

A CSP is defined by:

- Variables : $X, Y, Z \dots$
- Domains : Values for each variable.
- Constraints : Rules that must be satisfied.

Example: Map Coloring

- Variables : WA, NT, SA, Q, NSW, V.
- Domain : {Red, Green, Blue}
- Constraints : Adjacent regions cannot have the same color.



5. Means-End Analysis

It is a problem-solving technique that works backward from the goal to the current state.

Steps:

- ① Set the goal.
- ② Identify the differences between current state and goal.
- ③ Choose an operator to reduce the difference.
- ④ Repeat until the goal is reached.



6. Min-Max and Alpha-Beta Pruning

Min-Max Algorithm

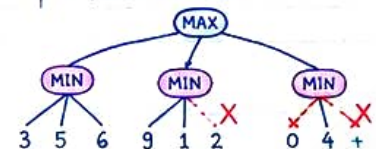
- Used in two-player games (like Tic-Tac-Toe, Chess).
- Maximizer tries to get highest score.
- Minimizer tries to get lowest score.
- Explores all possible moves.
- Time complexity: $O(b^d)$.

Example:

If a branch cannot produce a better result than the current best, it is pruned.

Alpha-Beta Pruning

- Optimizes Min-Max by cutting off unpromising branches.
- Uses two values:
 - Alpha (best already explored for maximizer)
 - Beta (best already explored for minimizer)
- Reduces search space without changing the result.
- Faster execution.



Key Takeaway : > Problem characteristics help us understand the nature of the problem.
> Search algorithms, CSP, means-end analysis and Min-Max with alpha-beta pruning are important techniques to solve AI problems efficiently.

Background of Artificial Intelligence

Unit - I
(112225)

1. Historical Background

- The idea of creating intelligent machines is not new. It has roots in ancient mythology and philosophy (e.g., Greek myths, mechanical automata).
- In 1950, Alan Turing proposed the Turing Test to check machine intelligence.
- In 1956, the term "Artificial Intelligence" was coined at the Dartmouth Conference (USA), which is considered the birth of AI as a field.

"AI is a field of study that aims to create machines capable of performing tasks that usually require human intelligence."

4. Key Influences and Drivers

- Technological Advances** - High-speed computing, better algorithms, more data.
- Data Availability** - Growth of digital data (internet, sensors, social media).
- Economic Demand** - Automation, efficiency, cost reduction.
- Scientific Research** - Advances in machine learning, neuroscience, cognitive science.
- Societal Needs** - Healthcare, education, security, smart living.
- Government & Industry Support** - Funding, policies, innovation labs.



2. Evolution of AI

- 1950** → Turing Test (Alan Turing)
- 1956** → Dartmouth Conference (term "Artificial Intelligence" coined)
- 1960s** → Early AI programs and symbolic reasoning (e.g., Logic Theorist).
- 1970s** → AI winter (reduced funding and interest).
- 1980s** → Expert systems and knowledge-based approaches.
- 1990s** → Machine learning and data-driven methods.
- 2000s** → Big data, cloud computing, deep learning.
- 2010s+** → Modern AI (deep learning, NLP, computer vision, generative AI).

5. AI and Related Fields

Field	Relation with AI
Computer Science	Provides programming, algorithms, data structures.
Mathematics	Used in logic, probability, statistics, optimization.
Cognitive Science	Helps in understanding human thinking and behavior.
Neuroscience	Inspires neural networks and learning models.
Psychology	Helps in human decision making and behavior.
Philosophy	Raises questions about intelligence, consciousness and ethics.
Linguistics	Supports Natural Language Processing (NLP).

3. Key Milestones

- 1950 - Turing Test
- 1956 - Dartmouth Conference
- 1966 - ELIZA (first chatbot)
- 1972 - PROLOG (logic programming)
- 1986 - Backpropagation (neural networks)
- 1997 - Deep Blue (beats world chess champion)
- 2006 - Deep Belief Networks
- 2012 - AlexNet (breakthrough in image recognition)
- 2016 - AlphaGo (beats Go champion)
- 2020+ - GPT, ChatGPT, Generative AI



From
Logic to
Learning

6. Present Scenario

- AI is now used in many real-world applications like healthcare, education, finance, transportation, agriculture, and entertainment.
- Deep learning and generative AI (like ChatGPT, DALL-E) are transforming industries.
- AI is moving from research labs to everyday life.

"AI is not just technology,
it is a global revolution."



7. Future Possibilities

- More human-like AI (stronger general intelligence).
- AI in robotics and autonomous systems.
- Better collaboration between humans and machines.
- Ethical and safe AI development.
- Solving global challenges (climate change, healthcare, etc.).

AI's journey is from imagination → to invention → to intelligence → to a better future.

Applications of AI and Turing Test

Unit - I
(112225)

1. Applications of AI

AI is used in many real-world applications to make our life easier, faster and more efficient. It helps in automation, decision-making, prediction and problem solving.

Major Areas of Applications

1. Healthcare

- Disease diagnosis (e.g., cancer detection).
- Medical image analysis (X-ray, MRI).
- Robotic surgery.
- Personalized treatment.



2. Education

- Personalized learning.
- Virtual tutors and chatbots.
- Automated grading.
- Adaptive learning systems.



3. Finance

- Fraud detection.
- Risk analysis and credit scoring.
- Algorithmic trading.
- Automated customer service (chatbots).



4. Transportation

- Self-driving cars.
- Traffic management.
- Route optimization.
- Smart transportation systems.



5. Agriculture

- Crop monitoring (drones & sensors).
- Disease detection in plants.
- Yield prediction.
- Smart irrigation systems.



6. Business & Industry

- Process automation.
- Inventory management.
- Robotics in manufacturing.
- Customer behavior analysis.



7. Entertainment & Media

- Recommendation systems (Netflix, YouTube).
- Game playing (AI bots).
- Content creation (text, image, music).
- Virtual assistants (Siri, Alexa).



8. Defence & Security

- Surveillance and monitoring.
- Threat detection.
- Autonomous weapons.
- Cybersecurity (intrusion detection).



AI is everywhere – from smartphones to space missions!

2. Turing Test

The Turing Test is a method proposed by Alan Turing in 1950 to determine whether a machine can exhibit intelligent behavior equivalent to a human being.

“If a machine can convince a human that it is also a human, then it is said to have passed the Turing Test.”

– Alan Turing

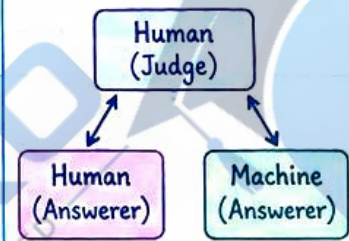


Alan Turing
(1912 – 1954)

How It Works?

- 1 A human (judge) has a conversation with two entities – one is a human and the other is a machine.
- 2 The judge does not know which is which.
- 3 The judge asks questions (text-based) and tries to determine the identity of each entity.
- 4 If the judge cannot reliably tell the machine from the human (even after enough questions), the machine is said to pass the test.

The Turing Test Setup



Key Points

- Proposed by: Alan Turing (1950).
- Purpose: To test machine intelligence through natural language.
- Method: Human evaluator converses with unknown participant (human or machine).
- Criterion: If the evaluator cannot distinguish, the machine is considered intelligent (has passed the test).
- It focuses on behavior, not internal code or mechanism.

Why It Matters?

- It was one of the first formal tests of AI.
- It changed the way we think about intelligence.
- Still a widely used concept in AI research and discussions.

Limitations

- It only tests conversational ability, not real intelligence.
- A machine might pass the test without understanding (just good imitation).
- It does not measure creativity, emotion, or real-world problem solving.
- Human judgment can be biased.

Turing Test doesn't prove intelligence, it just tests the appearance of intelligence.

Artificial General Intelligence (AGI)

Unit - II
(112225)

1. What is AGI ?

- Artificial General Intelligence (AGI) refers to a type of AI that has the ability to understand, learn, reason, and apply knowledge across a wide range of tasks – just like a human being.
- AGI is not limited to a specific domain or task; it can perform any intellectual task that a human can do.

“AGI aims to build machines that can think, learn and adapt like humans.”

2. Key Characteristics of AGI

- Generalization** – can learn and apply knowledge across different domains.
- Adaptability** – handles new and unseen situations.
- Reasoning** – uses logic and critical thinking.
- Learning** – learns from experience, not just data.
- Autonomy** – can make decisions without human intervention.
- Transfer of knowledge** – applies what it learns in one area to another.

3. How AGI Differs from Narrow AI

Feature	Narrow AI (Weak AI)	AGI (Strong AI)
Scope	Task-specific	General-purpose
Learning	Limited to training data	Learns across domains
Reasoning	No deep reasoning	Human-like reasoning
Adaptability	Low	High
Examples	Siri, Alexa, image recognition, chatbots	Human-like AI assistant

4. Why is AGI Important ?

- Can solve complex real-world problems.
- Automates a wide range of jobs and tasks.
- Helps in scientific research and innovation.
- Improves healthcare, education, and agriculture.
- Can make human life easier and more efficient.
- Opens new possibilities in space exploration, robotics, and beyond.

“AGI has the potential to solve problems that are beyond human capability.”

5. Examples / Possibilities of AGI

- A robot that can do household work, drive a car, and also do medical diagnosis.
- An AI assistant that can learn any subject, like a human student.
- Self-improving systems that can design new AI models.
- Robots that can work in factories, homes, hospitals, and even creative fields (art, music, etc.).

6. Challenges in Building AGI

- Complexity of human brain and behaviour.
- Need for massive and diverse data.
- Safety and alignment with human values.
- Avoiding bias and ensuring fairness.
- High computational power and cost.
- Uncertainty in decision-making.
- Ethical and legal concerns.

7. AGI vs AI vs Human Intelligence

Aspect	AI (Narrow)	AGI	Human Intelligence
Scope	Specific tasks	All domains	All domains
Learning	From data	From experience + data	From experience, intuition, emotions
Reasoning	Limited	Advanced (human-like)	Creative & abstract thinking
Adaptability	Low	High	Very high
Emotions	No	May have (simulated)	Yes
Example	Chatbot, image classifier	Human-like AI assistant	You

8. Current Status & Future

- AGI is still a research goal, not yet achieved.
- Many companies (OpenAI, DeepMind, Google, etc.) are working towards it.
- Expected in the future (maybe in next 5-20 years).
- It will require breakthroughs in machine learning, brain science, and computing power.

9. Ethical Considerations

- Ensuring safety and control.
- Preventing misuse (e.g., weapons, cyber attacks).
- Protecting human jobs and society.
- Maintaining privacy and data security.
- Making sure AGI benefits all, not just a few.



AGI =
Human-like AI
for a better future

→ AGI is not just about smarter machines, it's about creating a future where AI can truly understand, learn and grow with us. ←

Super AI

(Artificial Superintelligence)

Unit - I
(112225)

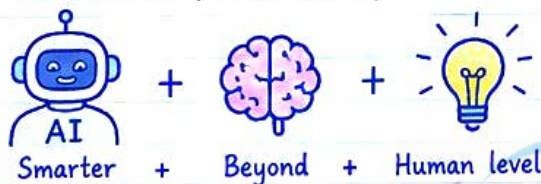
1. Introduction

- Super AI (Artificial Superintelligence) is a hypothetical form of AI that would surpass human intelligence across virtually all cognitive tasks.
- It goes beyond the capabilities of current AI systems and can think, learn, create and solve problems better than humans.
- It is not currently achieved and remains a future possibility, not a present reality.

"Super AI is not just smarter AI, it is intelligence beyond human level."

2. Key Features

- **Exceeds human intelligence** - can perform all cognitive tasks better than humans.
- **Self-improving** - learns and improves without human intervention.
- **Generalized ability** - can understand, learn and work in any domain.
- **Creativity** - can generate new ideas, solutions and innovations.
- **Emotional and social intelligence** - can understand and respond to human emotions and behaviour (advanced level).



3. Difference between Narrow AI, AGI and Super AI

Aspect	Narrow AI	AGI	Super AI
Meaning	Designed for specific tasks	Human-level intelligence across domains	Surpasses human intelligence in all domains
Capability	Task-specific	General intelligence	Far beyond human intelligence
Examples	Siri, Alexa, ChatGPT	Future systems (not yet achieved)	Hypothetical (not yet achieved)
Current status	Exists	Research stage	Not achieved

4. Advantages

- Can solve complex problems beyond human capability.
- Faster and more accurate decision-making.
- Accelerates scientific research and discovery.
- Improves productivity and efficiency in all fields.
- Can help in solving global challenges (climate change, healthcare, poverty, etc.).



5. Challenges and Risks

- Loss of human jobs due to automation.
- Ethical and moral concerns (bias, misuse).
- Risk of loss of control (unexpected behaviour).
- Privacy and security issues (data misuse).
- Potential for harm (in warfare or cyberattacks).
- Inequality and concentration of power.



6. Present / Future Possibilities

Present Possibilities

- Improved human-computer interaction (more natural and personalized).
- Advanced research in medicine, education and science.
- Better problem-solving and decision support systems.



Future Possibilities

- Solving global challenges (climate change, healthcare, food security).
- Autonomous systems and smart cities.
- New forms of creativity and innovation.
- Potential to enhance human life and expand our understanding of the universe.

7. Conclusion

- Super AI has the potential to bring tremendous benefits to humanity, but it also comes with serious risks.
- It is not currently achieved and may take a long time to develop.
- Responsible research, strong ethical guidelines and global cooperation are essential to ensure that Super AI is safe, fair and beneficial for all.

The goal should be not just to create Super AI, but to create a better future with it.

— Super AI : A step beyond human intelligence —

Rational Agent Approach to AI

Unit - I
(112225)

1. Introduction

- The Rational Agent Approach is one of the most widely accepted perspectives in Artificial Intelligence.
- It focuses on designing systems (agents) that act rationally, i.e., make the best decision that maximizes their chances of success.
- A rational agent is not just intelligent, but also goal-oriented and performance-driven.

"A rational agent is one that takes actions that are expected to maximize its performance measure, given its percepts and knowledge."

4. How it Works

- ① The agent senses the environment and receives percepts (information).
- ② It uses its knowledge and reasoning to evaluate possible actions.
- ③ It selects the action that is expected to give the highest performance measure.
- ④ The action is performed through actuators.
- ⑤ The process continues repeatedly.

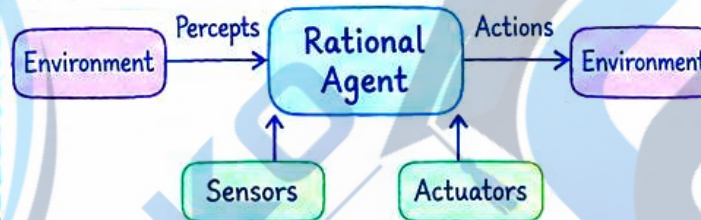
8. Conclusion

☆ The Rational Agent Approach is powerful because it focuses on achieving the best possible outcome. It forms the foundation for many modern AI systems, from search engines to self-driving cars, making AI more useful, reliable and goal-oriented. ☆

2. What is a Rational Agent?

- A rational agent perceives its environment through sensors, processes the information, and takes actions through actuators.
- It tries to choose the action that gives the highest expected performance, using a set of knowledge and rules.

Basic Structure of a Rational Agent



5. Example

In a self-driving car:

- Sensors collect data (camera, radar).
- The agent processes the data and decides the best action (e.g., turn, stop, accelerate).
- Actuators control the car's movement.
- The goal is safety, so it chooses the action that gives the best result.

3. Key Components

- ① **Performance Measure** - defines what "success" means (e.g., accuracy, profit, safety).
- ② **Environment** - everything outside the agent (including other agents).
- ③ **Sensors** - get information from the environment (e.g., camera, microphone).
- ④ **Actuators** - perform actions in the environment (e.g., motor, display).
- ⑤ **Knowledge** - internal information, rules, and experience to make decisions.

6. Advantages

- Helps in building intelligent and autonomous systems.
- Provides a clear framework for decision-making.
- Works well in dynamic and uncertain environments.
- Can handle complex real-world problems.

7. Limitations

- Designing a good performance measure is difficult.
- May not work well in highly unpredictable situations.
- Requires more computation and resources.
- Does not always consider ethical or human values.

Intelligent Agents and Structure of Intelligent Agents

Unit - I
(112225)

1. Intelligent Agents

- An intelligent agent is a system that perceives its environment through sensors and takes actions through actuators in order to achieve a goal.
- It acts autonomously, meaning it makes decisions on its own based on the information it receives.
- Intelligent agents are designed to be rational, i.e., they try to select the best action to achieve the desired goal.

"An intelligent agent is an entity that perceives its environment and acts upon it to achieve its goals."

2. Key Features

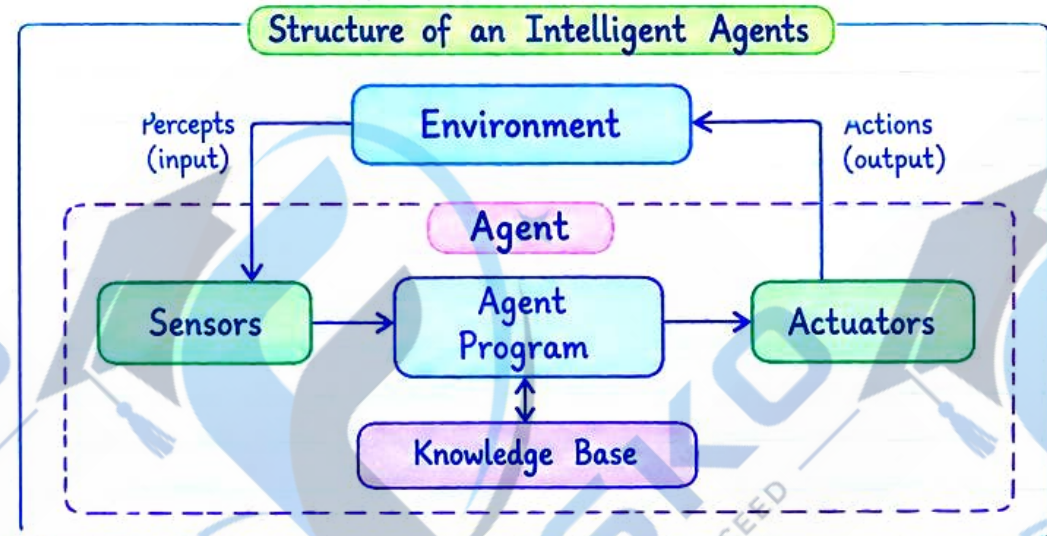
- **Autonomy** – works without human intervention.
- **Perception** – gathers information from the environment.
- **Rationality** – takes the best action to achieve goals.
- **Reactivity** – responds to changes in the environment.
- **Proactiveness** – takes initiative to achieve goals.
- **Learning** – improves performance over time.

3. Types of Intelligent Agents

- ① **Simple Reflex Agents** – act only on current percepts.
- ② **Model-based Reflex Agents** – maintain internal state of the world.
- ③ **Goal-based Agents** – act to achieve specific goals.
- ④ **Utility-based Agents** – choose actions that maximize utility.
- ⑤ **Learning Agents** – improve with experience and learning.

4. Structure of Intelligent Agents

- The structure of an intelligent agent shows how it works and interacts with its environment.



Components

- ① **Sensors** – collect information from the environment (e.g., camera, microphone, temperature sensor).
- ② **Agent Program** – processes the information, applies logic and decides what action to take.
- ③ **Actuators** – perform actions in the environment (e.g., move a robot, show output, control a device).
- ④ **Knowledge Base** – stores knowledge, facts and rules about the environment and itself.



The structure helps the agent to perceive, think, learn and act in the environment.

Behaviour of Intelligent Agents

Unit - I
(112225)

1. Introduction

- The behaviour of an intelligent agent refers to how it perceives its environment, makes decisions and takes actions to achieve its goals.
- It is the result of the agent's internal program, knowledge and learning process.
- The behaviour depends on the type of agent, its goals, and the environment it operates in.

"An intelligent agent behaves rationally to achieve its goals in a given environment."

2. Components Affecting Behaviour

- **Perception** - receives information from the environment (e.g., sensors).
- **Knowledge** - stores facts and rules about the world.
- **Reasoning** - processes information and makes decisions.
- **Learning** - improves behaviour from experience.
- **Actions** - performs tasks through actuators.

6. Key Points

- ✓ Behaviour is the result of perception, reasoning and action.
- ✓ Different agents show different types of behaviour.
- ✓ The environment and goals strongly influence behaviour.
- ✓ Learning helps agents improve over time.

3. Types of Behaviour

① Reactive Behaviour

- Responds directly to current perception.
- No internal model or memory.
- Example: Reflex agents (e.g., thermostat).

② Goal-based Behaviour

- Takes actions to achieve specific goals.
- Uses planning and reasoning.
- Example: Robot moving to a target location.

③ Utility-based Behaviour

- Chooses the best action based on a utility function.
- Handles multiple goals and trade-offs.
- Example: Self-driving car (balances safety, speed, comfort).

④ Learning Behaviour

- Improves over time using experience.
- Adapts to new situations.
- Example: Machine learning systems (e.g., recommendation).

7. Real-World Examples

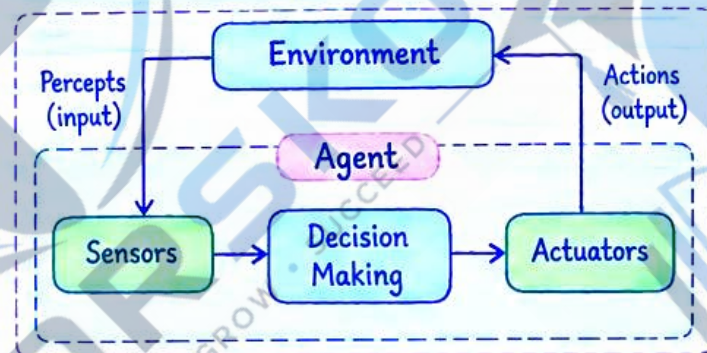
- **Reflex Agent** - Thermostat (temperature control).
- **Goal-based Agent** - Robot (reach a destination).
- **Utility-based Agent** - Shopping app (best price + quality).
- **Learning Agent** - Netflix (better recommendations).

4. Factors Influencing Behaviour

- **Goals** - what the agent wants to achieve.
- **Environment** - changes and uncertainties.
- **Knowledge** - what the agent knows.
- **Sensors & Actuators** - input and output devices.
- **Learning** - adapts from past experience.

5. Example - Behaviour in a Self-Driving Car

The car perceives the environment (camera, radar), uses AI to decide the best action (brake, turn, accelerate) and takes action through actuators (steering, brakes).



8. Conclusion

The behaviour of intelligent agents is goal-driven, adaptive and rational. It depends on perception, knowledge, reasoning, learning and the environment. This behaviour makes AI agents useful in real-world applications.



Better Perception + Smarter Decisions + Meaningful Actions = Intelligent Behaviour

1. Definition of Task Environment

- A task environment is the external world in which an intelligent agent operates. It defines the situation, the available information, and the goals or objectives the agent must achieve.

"Task environment specifies what the agent perceives, what it can do, and what it aims to achieve."

2. PEAS: Components

- Performance Measure** :
- Defines how well the agent performs.
 - e.g., success rate, time, safety, efficiency, reward, etc.
- Environment** :
- The external world the agent operates in.
 - e.g., physical world, people, objects, other agents, etc.
- Actuators** :
- Devices used to take actions in the environment.
 - e.g., wheels, motors, steering, display, speakers, etc.
- Sensors** :
- Devices used to perceive the environment.
 - e.g., camera, lidar, GPS, microphone, temperature sensor, etc.

3. PEAS Example: Self-Driving Car

Component	Example
Performance Measure	Safety, reach destination, minimum time, low energy consumption, smooth ride.
Environment	Roads, traffic, pedestrians, other vehicles, weather, traffic signals, lanes, buildings.
Actuators	Steering, accelerator, brakes, indicators, horn, lights.
Sensors	Camera, lidar, GPS, radar, ultrasonic sensors, IMU, microphones.

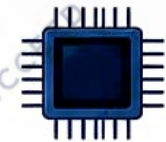
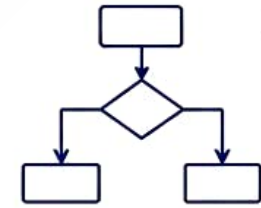
4. Agent - Environment Interaction



5. Properties of Task Environments

- Fully / Partially Observable** : Agent can / cannot perceive the complete state.
- Deterministic / Stochastic** : Actions have definite / probabilistic outcomes.
- Episodic / Sequential** : Each task is independent / depends on previous actions.
- Static / Dynamic** : Environment does / does not change while the agent acts.
- Discrete / Continuous** : States, actions and time are discrete / continuous.
- Single / Multi-agent** : One agent / multiple agents exist in the environment.

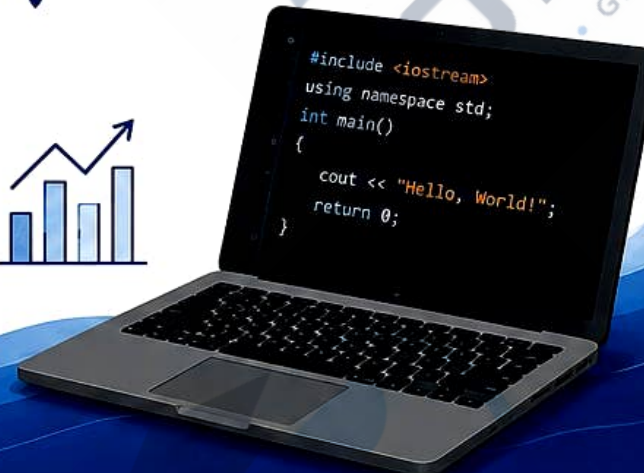
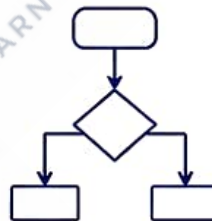
1010101010
0101010101
1010101010
0101010101



Unit - II



10110
01001
10101



Problem Characteristics

Unit - II
(112225)

1. Introduction

- Problem characteristics describe the features or properties of a problem that help in selecting the most suitable problem solving technique or strategy.
- Different problems have different nature, and hence require different approaches for solution.

2. Key Characteristics of a Problem

1. Well-defined / Ill-defined :

- Well-defined problems have clear goals, initial state and solution path (e.g., mathematical problems).
- Ill-defined problems have vague goals or incomplete information (e.g., real-world problems).

2. Deterministic / Non-deterministic :

- Deterministic problems have a definite outcome (for a given input).
- Non-deterministic problems have multiple possible outcomes (e.g., games, unpredictable environments).

3. Static / Dynamic :

- Static problems do not change over time.
- Dynamic problems change with time (e.g., robot navigation).

4. Fully Observable / Partially Observable :

- Fully observable problems have complete information about the environment.
- Partially observable problems have incomplete information (e.g., sensors with limited range).

5. Single-agent / Multi-agent :

- Single-agent problems are solved by one agent (e.g., puzzle solving).
- Multi-agent problems involve multiple agents with different goals (e.g., robotics, game playing).

6. Continuous / Discrete

- Continuous problems have variables in a continuous range (e.g., temperature control).
- Discrete problems have distinct, separate values (e.g., chess, scheduling).

7. Sequential / Episodic :

- Sequential problems involve a series of actions (e.g., planning, route finding).
- Episodic problems are independent of previous actions (e.g., classification, image recognition).

4. Importance

- Helps in choosing the right AI technique (e.g., search, planning, machine learning).
- Improves efficiency and accuracy of problem solving.
- Helps in designing suitable algorithms and models.

3. Examples

- Finding the shortest path in a map
→ well-defined, deterministic, static, discrete, single-agent.
- Playing chess
→ well-defined, deterministic, sequential, discrete, multi-agent.
- Weather prediction
→ ill-defined, non-deterministic, dynamic, continuous.
- Self-driving car
→ partially observable, dynamic, continuous, multi-agent.

"Understanding the characteristics of a problem is the first step towards an effective solution."



Good Problem Analysis → Better Strategy → Smarter AI Solutions

Production Systems

Unit - II
(112225)

1. Introduction

- A production system is a problem-solving model that represents knowledge in the form of production rules and uses a control strategy to solve problems by applying these rules.
- It is widely used in AI for symbolic reasoning, expert systems, and automated problem solving.

"A production system consists of a set of rules and a control strategy to apply the rules."

2. Components of a Production System

1. Knowledge Base

- Contains a set of production rules.
- Represents the knowledge about the problem domain.

2. Working Memory (Fact Base)

- Stores the current state of the problem.
- Contains facts or data that are updated during problem solving.

3. Inference Engine (Control Strategy)

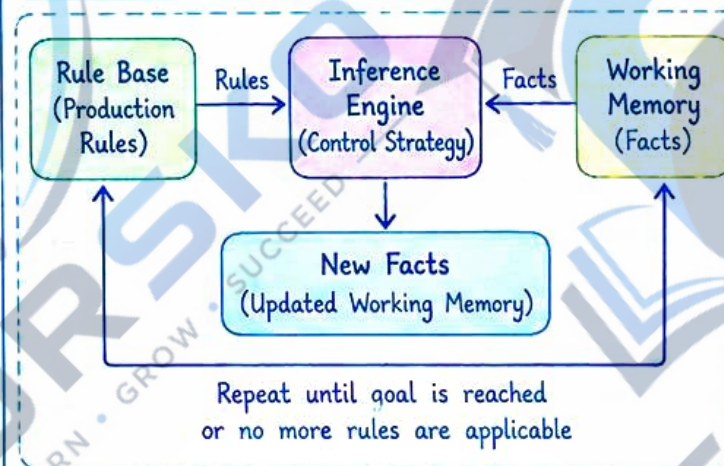
- Matches rules with facts in working memory.
- Selects and applies the appropriate rule.

4. Rule Base

- Collection of IF - THEN rules.
- Defines how to transform known facts into new facts.

3. Working of a Production System

- The facts in the working memory are matched with the conditions (IF part) of the rules.
- The inference engine selects a rule (based on strategy like conflict resolution).
- The action part (THEN part) of the rule is executed.
- New facts are added to the working memory and the process continues until the goal is reached or no more rules can be applied.



4. Control Strategies

- Forward Chaining** - Start with known facts and infer new facts (data-driven).
- Backward Chaining** - Start with goal and work backwards to find supporting facts (goal-driven).

5. Types of Production Systems

1. Based on Control Strategy

- Forward Chaining (data driven)
- Backward Chaining (goal driven)

2. Based on Knowledge Representation

- Rule-based systems
- Frame-based systems
- Object-oriented production systems



IF ... THEN ...
(Production Rule)

6. Advantages

- Simple and easy to understand.
- Modular and flexible.
- Allows separation of knowledge and control.
- Widely used in expert systems.

7. Limitations

- Rule explosion (large number of rules).
- Not suitable for complex, uncertain problems.
- Requires efficient rule selection strategy.
- Maintenance can be difficult for large systems.

Example

In a medical diagnosis system:

IF fever AND cough THEN flu.

IF fever AND rash THEN measles.

IF headache AND stiffness THEN meningitis.

Production systems → Knowledge + Rules + Control → Intelligent problem solving

Control Strategies

Unit - II
(112225)

1. Introduction

- Control strategies are used to guide and manage the problem-solving process in AI.
- They determine how the system searches, selects and applies knowledge to reach the goal.
- Control strategies play a key role in deciding the order of operations and the use of rules in problem solving.

"Control strategy is the mechanism that decides which rule or operation to apply next."

2. Types of Control Strategies

1. Linear Control Strategy

- Rules are applied in a fixed, predetermined order.
- Simple but not flexible.

2. Heuristic Control Strategy

- Uses domain knowledge and experience to guide the search.
- More efficient than linear strategy.

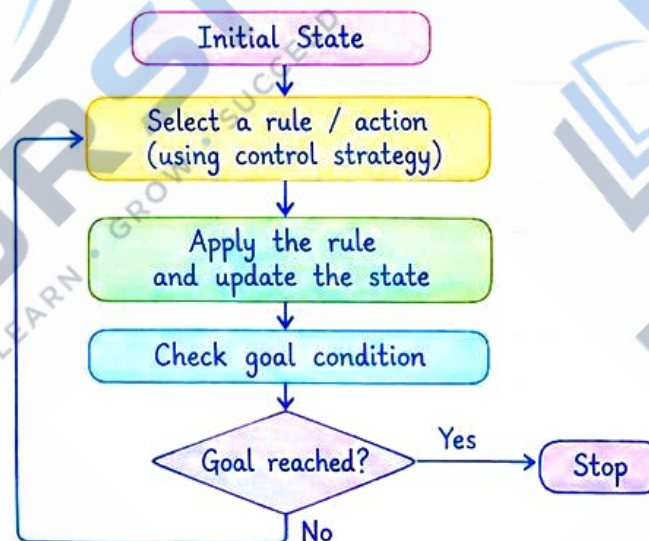
3. Goal-Oriented Control Strategy

- Selects actions that bring the system closer to the goal.
- Uses evaluation functions or heuristics.

4. Comparison of Control Strategies

Feature	Linear	Heuristic	Goal-Oriented
Rule Order	Fixed	Based on knowledge	Based on goal
Flexibility	Low	Medium	High
Efficiency	Low	High	Very high
Use of Heuristics	No	Yes	Yes
Example	Sequential search	Best-first search	A* algorithm

5. Working of Control Strategies



6. Examples

1. Linear Control Strategy

- Example: Depth-first search (fixed order of rules).
- Useful in simple and well-defined problems.

2. Heuristic Control Strategy

- Example: Best-first search (uses heuristic function).
- Helps in reducing search time and effort.

3. Goal-Oriented Control Strategy

- Example: A* algorithm (uses cost + heuristic).
- Finds optimal solution efficiently.

7. Importance

- Improves the efficiency of problem solving.
- Reduces search space and time.
- Helps in selecting the most appropriate actions.
- Essential for intelligent and autonomous behavior of AI systems.



Key Takeaway

"Control strategies decide the direction of problem solving and ensure the system reaches the goal efficiently."



Better control strategies → Faster search → Smarter AI systems

Breadth-First Search (BFS)

Unit - I
(112225)

1. Introduction

- Breadth-First Search (BFS) is a graph traversal algorithm that visits the nodes level by level, starting from a given source node.
- It explores all the neighbours of a node before moving to the next level.
- BFS is used for searching, shortest path (in unweighted graphs), and finding connected components.

"BFS explores the graph layer by layer using a queue."

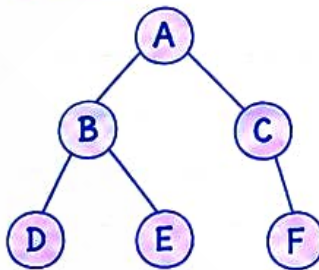
2. Key Features

- Uses a queue (FIFO) data structure.
- Visits nodes in level order.
- Finds shortest path in unweighted graphs.
- Guarantees minimum number of edges from source to all reachable nodes.
- Works for both directed and undirected graphs.

3. Algorithm Steps

- Start from the source node and mark it as visited.
- Enqueue the source node.
- While the queue is not empty:
 - Dequeue a node and visit it.
 - Add all unvisited neighbours to the queue.
 - Mark them as visited.

4. Example Graph



BFS Traversal (from A)

Level 0 : A

Level 1 : B, C

Level 2 : D, E, F

Traversal Order:

A → B → C → D → E → F

5. Pseudocode

BFS(G, s)

- create empty queue Q
- mark s as visited
- enqueue(Q, s)
- while (Q is not empty) do
- v ← dequeue(Q)
- visit(v)
- for each neighbour u of v do
- if (u is not visited) then
- mark u as visited
- enqueue(Q, u)

6. BFS vs DFS

Feature	BFS	DFS
Traversal Order	Level by level	Depth first
Data Structure	Queue	Stack (or recursion)
Shortest Path (unweighted)	Yes	No
Memory Usage	More	Less
Use Cases	Shortest path, connectivity	Topological sort, cycle detection

7. Applications

- Finding shortest path in unweighted graphs.
- Network broadcasting and routing.
- Connected components in a graph.
- Social network analysis.
- Web crawling.

8. Time and Space Complexity

- Time Complexity : $O(V + E)$
(V = number of vertices, E = number of edges)
- Space Complexity : $O(V)$
(for visited array and queue)



BFS is simple, efficient and widely used for level-wise exploration of graphs.

Depth-First Search (DFS)

Unit - I
(112225)

1. Introduction

- Depth-First Search (DFS) is a graph traversal algorithm that explores as far as possible along each branch before backtracking.
- It uses a stack (explicit or implicit) to keep track of the next node to visit.
- DFS is widely used in problems like topological sorting, cycle detection, and connected components.

"DFS goes deep first, then backtracks to explore other paths."

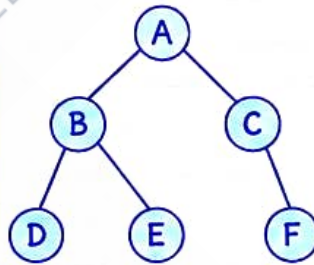
2. Key Features

- Uses a stack (or recursion) for traversal.
- Visits nodes in depth-first order.
- Explores a branch completely before moving to the next branch.
- Can be implemented using recursion or an explicit stack.
- Useful for finding connected components, paths, and topological sorting.

3. Algorithm Steps

- Start from the source node (root).
- Mark the node as visited.
- Visit all unvisited neighbours (go deeper).
- If no unvisited neighbour exists, backtrack and explore the next unvisited node.

4. Example Graph



DFS Traversal (from A)

A → B → D → E → C → F

(One possible DFS order.
Order may vary depending
on the starting node and
neighbour selection.)

5. Pseudocode

DFS(G, v)

- mark v as visited
- print(v)
- for each neighbour u of v do
- if (u is not visited) then
- DFS(G, u)

6. Recursive Implementation

```
void DFS(int v)
{
    visited[v] = true;
    printf("%d ", v);
    for (each neighbour u of v)
        if (!visited[u])
            DFS(u);
}
```

7. DFS vs BFS

Feature	DFS	BFS
Traversal Order	Depth first	Level by level
Data Structure	Stack (or recursion)	Queue
Shortest Path	Not guaranteed	Guaranteed (for unweighted graphs)
Memory Usage	Less	More
Use Cases	Topological sort, cycle detection, connected components	Shortest path, level order traversal

8. Applications

- Finding connected components in a graph.
- Detecting cycles in a graph.
- Topological sorting.
- Solving puzzles and maze problems.
- Web crawling.

9. Advantages and Disadvantages

Advantages

- Simple to implement.
- Uses less memory (than BFS).
- Effective for many real-world problems.

Disadvantages

- Does not guarantee shortest path.
- Can get stuck in infinite loops (if not handled properly).



DFS is simple, memory efficient
and goes deep into the graph!

DFS explores → goes deep → backtracks → finds the solution

Hill Climbing

1. Introduction

- Hill climbing is a local search algorithm used to find a solution to an optimization problem.
- It starts from an initial state and moves to a neighboring state with a better value (higher or lower than the current state).
- It continues until no better neighbor is found (or a goal state is reached).

"Hill climbing is a simple but effective technique for finding a local optimum."

2. Working / Procedure

- Start from an initial state.
- Evaluate the current state and its neighbours.
- Move to the best neighbour (improvement in value).
- Repeat until no better neighbour is found (or goal is reached).

Example (Maximization)



3. Key Features

- Simple and easy to implement.
- Uses only local information (no need to explore entire search space).
- Memory efficient (stores only current state).
- May get stuck in local maxima, plateaus or ridges.

4. Advantages

- Low memory requirement.
- Fast and simple.
- Works well for large problem spaces.
- Suitable for real-time applications.

5. Disadvantages

- Can get stuck in local maxima.
- May fail in plateaus (no improvement).
- No guarantee of finding the global optimum.
- Performance depends on the initial state and neighbour selection.

6. Applications

- Optimization problems (e.g., scheduling, routing).
- Robotics (navigation).
- Game playing (e.g., chess).
- Machine learning (feature selection).

Variations of Hill Climbing

- To overcome the limitations of simple hill climbing, several variations have been developed.

1. Steepest Ascent Hill Climbing (Best-First)

- Examines all neighbours and moves to the one with the highest improvement.
- Provides better results than simple hill climbing.
- Example: N-Queens problem.

2. First Choice Hill Climbing

- Randomly selects a neighbour and moves if it is better.
- Faster, but may miss the best move.
- Example: Large search spaces.

3. Stochastic Hill Climbing

- Chooses a random neighbour from the set of improving neighbours.
- Helps in escaping local maxima.
- Example: Continuous optimization problems.

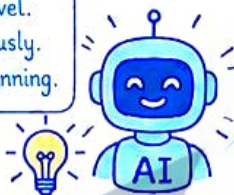
4. Hill Climbing with Random Restarts

- If stuck in a local maximum, restart from a random state.
- Increases the chance of finding the global optimum.
- Example: Complex optimization problems.

5. Beam Search (Limited Width)

- Keeps multiple states (beam) at each level.
- Explores more than one path simultaneously.
- Example: Natural language processing, planning.

Hill Climbing → Finds better solutions step by step → Variations improve performance.



Heuristic Search Techniques

Unit - I
(112225)

1. Introduction

- Heuristic search techniques use problem-specific knowledge (heuristics) to guide the search process towards the goal.
- They help to reduce the search space and improve efficiency compared to uninformed search.
- Heuristics are often based on estimates of the cost or distance to the goal.

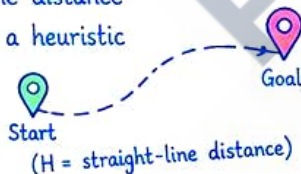
"Heuristic search combines the problem knowledge with search algorithms to find good solutions faster."

2. Key Features

- Uses heuristic (domain-specific) information.
- Guides the search towards promising states.
- Reduces the number of nodes explored.
- More efficient than uninformed search.
- Not always guarantees the optimal solution (except A* with admissible heuristic).

Heuristics Example

In a map, the straight-line distance to the goal can be used as a heuristic to guide the search.



3. Best-First Search

- Best-First Search expands the node that appears to be the most promising, based on a heuristic evaluation function.

Types

1. Greedy Best-First Search

- Uses only the heuristic function $h(n)$.
- Expands the node with the smallest $h(n)$.

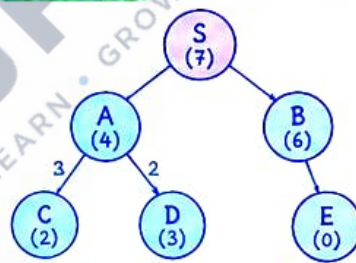
2. Uniform-Cost Search (a special case)

- Uses only the path cost $g(n)$.
- Expands the node with the smallest $g(n)$.

Steps (Greedy Best-First Search)

- Put the start node in the priority queue.
- Expand the node with the lowest $h(n)$.
- Add its successors with their $h(n)$ values.
- Repeat until the goal is found.

Example



Node	$h(n)$
S	7
A	4
B	6
C	2
D	3
E	0

Greedy Best-First Search expands the node with the smallest $h(n)$.

4. A* Algorithm

- A* combines the advantages of Uniform-Cost Search and Greedy Best-First Search.
- It uses both the path cost and heuristic value.
- It always finds the optimal solution (if the heuristic is admissible and consistent).

Evaluation Function

$$f(n) = g(n) + h(n)$$

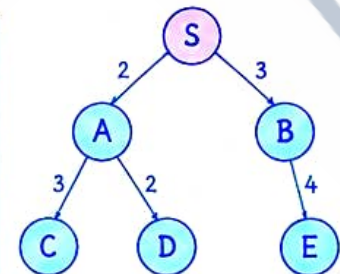
$g(n)$ = cost from start to n
 $h(n)$ = estimated cost from n to goal
 $f(n)$ = total estimated cost

Steps

- Put the start node in the priority queue.
- Expand the node with the smallest $f(n)$.
- Update the costs of successors and their $f(n)$ values.
- Repeat until the goal node is reached.

Example

Node	$g(n)$	$h(n)$	$f(n)$
S	0	7	7
A	2	4	6
B	3	6	9
C	5	2	7
D	5	3	8
E	7	0	7



A* expands nodes in the order of lowest $f(n)$.

Heuristic Search → Uses problem knowledge → More efficient search → (A* gives optimal solution)

Constraint Satisfaction Problem (CSP)

Unit - II
(112225)

1. Introduction

- A Constraint Satisfaction Problem (CSP) is a problem of finding values for a set of variables such that all the given constraints are satisfied.
- It is widely used in AI for solving scheduling, planning, configuration and resource allocation problems.
- CSPs are common in real-world problems like map coloring, timetabling, Sudoku, and puzzle solving.

"A CSP is defined by three components: variables, domains and constraints."

2. Components of a CSP

- Variables** - The decision points (e.g., states, time slots, rooms).
- Domains** - The possible values each variable can take (e.g., colors, numbers, time slots).
- Constraints** - Rules that restrict the values (e.g., neighbouring regions cannot have the same color).

Example (Map Coloring)

- Variables** : WA, NT, SA, Q, NSW, V
- Domain** : {Red, Green, Blue}
- Constraint** : Adjacent regions cannot have the same color.

3. Formal Definition

A CSP is defined as a triple (X, D, C) where:

- X** = set of variables $\{X_1, X_2, \dots, X_n\}$
- D** = set of domains $\{D_1, D_2, \dots, D_n\}$
- C** = set of constraints $\{C_1, C_2, \dots, C_m\}$

A **solution** is an assignment of values to all variables such that all constraints are satisfied.

4. Types of Constraints

Type	Description	Example
Unary	Constraint on a single variable.	$X = \text{Red}$
Binary	Constraint between two variables.	$X \neq Y$
N-ary	Constraint between more than two variables.	All different (X, Y, Z)
Global	Applies to a set of variables (e.g., all-different).	All Different constraint

5. Solving CSPs

- Backtracking Search** - simple but effective.
- Forward Checking** - reduces future failures.
- Arc Consistency (AC-3)** - removes inconsistent values.
- Heuristics** - Minimum Remaining Values (MRV), Degree Heuristic, Least Constraining Value (LCV).

6. Example: Map Coloring



Constraint:

Adjacent regions cannot have the same color.

Possible solution:

- WA - Red
- NT - Green
- SA - Blue
- Q - Red
- NSW - Green
- V - Red

7. Applications

- Scheduling (e.g., exam timetabling).
- Resource allocation (e.g., assigning tasks).
- Planning (e.g., route planning).
- Configuration (e.g., product design).
- Games and puzzles (e.g., Sudoku, map coloring).

8. Conclusion

- CSP provides a powerful framework for solving problems with multiple constraints.
- It helps in finding valid solutions efficiently using search and inference techniques.
- It is widely used in many **AI applications** and real-life scenarios.

CSP = Variables + Domains + Constraints $\rightarrow$ Valid Solution

Means-End Analysis

Unit - II
(112225)

1. Introduction

- Means-End Analysis (MEA) is a problem-solving technique used in Artificial Intelligence to achieve a desired goal by reducing the difference between the current state and the goal state.
- It works by finding a sequence of intermediate subgoals (means) that lead from the current state to the goal state (end).
- It is a top-down, goal-directed approach.

"Means-End Analysis solves a problem by working backward from the goal to the current state, setting subgoals along the way."

2. Key Concepts

- Goal** - The desired final state.
- Current State** - The present situation.
- Subgoal** - An intermediate step towards the goal.
- Operators** - Actions that can be performed to achieve subgoals.
- Difference** - The gap between the current state and the goal state.

3. Steps in Means-End Analysis

- Set the goal** - Define the desired state.
- Compare with current state** - Find the differences between the current state and goal.
- Choose an operator** - Select an action that reduces the difference.
- Set a subgoal** - Create an intermediate goal that moves towards the main goal.
- Execute the operator** - Perform the action and update the current state.
- Repeat** - Continue the process until the goal is reached.

4. Advantages

- Simple and easy to understand.
- Works well for well-defined problems.
- Can be combined with other techniques (e.g., heuristic search).
- Reduces the problem into smaller subproblems.

5. Limitations

- Not suitable for complex or unclear goals.
- May require a large number of steps.
- Does not work well in dynamic environments (where the situation keeps changing).

6. Example

Problem: Reach the goal of getting the key using means-end analysis.

Initial State: You are in a room, the key is in another room, and the door is closed.

Goal: Get the key.

Step-by-step Process



Analysis:

- Current state:** You are in room 1, key in room 2, door closed.
- Subgoal 1:** Open the door.
- Subgoal 2:** Go to room 2.
- Subgoal 3:** Pick up the key.
- Goal reached:** You have the key.

7. Real-Life Applications

- Robotics (e.g., picking up an object).
- Game playing (e.g., chess, puzzle solving).
- Planning (e.g., travel route, task scheduling).
- Everyday decision making (e.g., reaching a destination).



Means-End Analysis helps an agent achieve its goal by breaking it into smaller, achievable steps.



Goal
↓
Subgoals
↓
Success

Minimax Algorithm

Unit - II
(112225)

1. Introduction

- Minimax algorithm is a decision-making algorithm used in two-player, zero-sum games (e.g., Tic-Tac-Toe, Chess, Checkers).
- It helps the agent (player) choose the best move, assuming the opponent also plays optimally.
- The algorithm works by exploring the game tree and evaluating possible moves using a utility function.

"Minimax assumes that the opponent will always play the best possible move for them (to minimize our gain)."

2. Key Concepts

- Game Tree** - Represents all possible moves and outcomes.
- Terminal State** - End of the game (win, lose or draw).
- Utility Function** - Assigns a value to each terminal state (e.g., +1, 0, -1).
- Maximizing Player** - Tries to get the highest score (our agent).
- Minimizing Player** - Tries to get the lowest score (opponent).
- Backtracking** - Used to evaluate values from bottom to top.

3. How it Works

- Start from the current state (root).
- Generate all possible moves (successor states).
- Evaluate terminal states using the utility function.
- Propagate the values upward:
 - Max node → choose the maximum value.
 - Min node → choose the minimum value.
- Return the best move for the current player.

4. Game Tree Structure



Values are backed up from the leaves to the root.

5. Characteristics

- Complete (if the tree is finite).
- Optimal (gives the best possible move).
- Time complexity: $O(b^d)$ (b = branching factor, d = depth).
- Space complexity: $O(b \times d)$ (with depth-first search).

6. Example - Tic Tac Toe

Consider a simple game state:

X	O	X
O	X	
	O	

X (Max) tries to win
O (Min) tries to block

The algorithm explores all possible moves, assigns utility values to outcomes, and chooses the move with the highest value (for X).

7. Advantages and Disadvantages

Advantages

- ✓ Guarantees the best move (if both play optimally).
- ✓ Works well for small to medium-sized games.
- ✓ Simple and easy to understand.

Disadvantages

- ✗ Time and space complexity is high (exponential).
- ✗ Becomes impractical for large games.
- ✗ Does not work well with uncertainty (e.g., stochastic environments).

8. Real-World Applications

- Chess and Checkers
- Tic-Tac-Toe
- Game playing AIs
- Decision making in robotics



Minimax Algorithm → Explore → Evaluate → Choose Best Move

Alpha Beta Pruning

Unit - II
(112225)

1. Introduction

- Alpha Beta Pruning is an optimization technique used in the Minimax algorithm to reduce the number of nodes that need to be evaluated in a game tree.
- It does not change the final result, but it makes the search process faster and more efficient.

"Alpha Beta Pruning avoids exploring branches that cannot influence the final decision."

2. Key Ideas

Alpha (α)

- Represents the best (highest) value MAX can guarantee so far.
- It is the lower bound for MAX.

Beta (β)

- Represents the best (lowest) value MIN can guarantee so far.
- It is the upper bound for MIN.

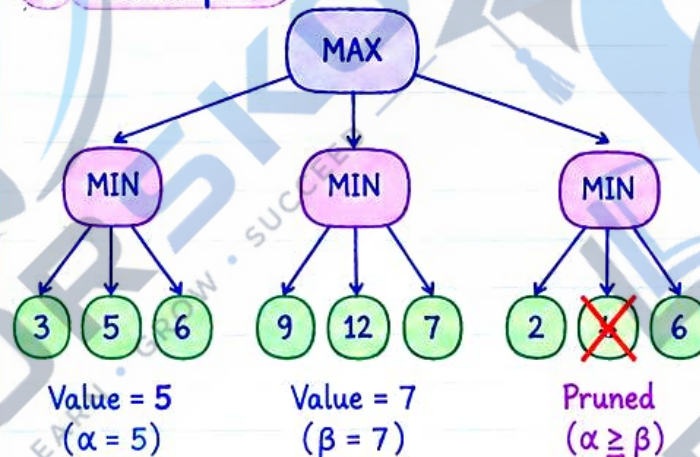
Pruning

- If a node's value cannot be better than the current α or β , that branch is not explored further.

3. How It Works

- Start with $\alpha = -\infty$ and $\beta = +\infty$.
- Traverse the game tree (usually depth-first).
- At MAX node: update $\alpha = \max(\alpha, \text{value})$.
- At MIN node: update $\beta = \min(\beta, \text{value})$.
- If $\alpha \geq \beta$, prune the remaining child nodes (no need to explore).
- Continue until the root node is evaluated.

4. Example



In the right subtree, the node with value 4 is not evaluated because $\alpha (5) \geq \beta (4)$.

5. Advantages

- Reduces the number of nodes to be evaluated.
- Speeds up the search process.
- Uses less memory (compared to other optimization techniques).
- Gives the same result as Minimax.

6. Disadvantages

- Slightly more complex to implement than Minimax.
- Worst-case time complexity is still the same as Minimax ($O(b^d)$).
- Does not change the final decision (only improves efficiency).

7. Comparison: Minimax vs Alpha Beta Pruning

Feature	Minimax	Alpha Beta Pruning
Result	Same	Same
Nodes evaluated	More	Fewer
Time complexity	$O(b^d)$	$O(b^d)$ (worst case)
Memory usage	More	Less
Efficiency	Lower	Higher

8. Conclusion

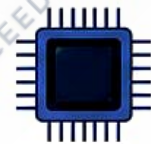
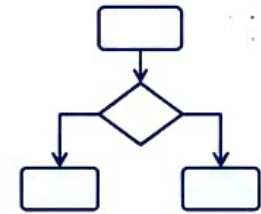
- Alpha Beta Pruning is a simple but powerful technique that makes the Minimax algorithm faster and more efficient, especially in large games.



Same Decision + Fewer Nodes = Faster Search



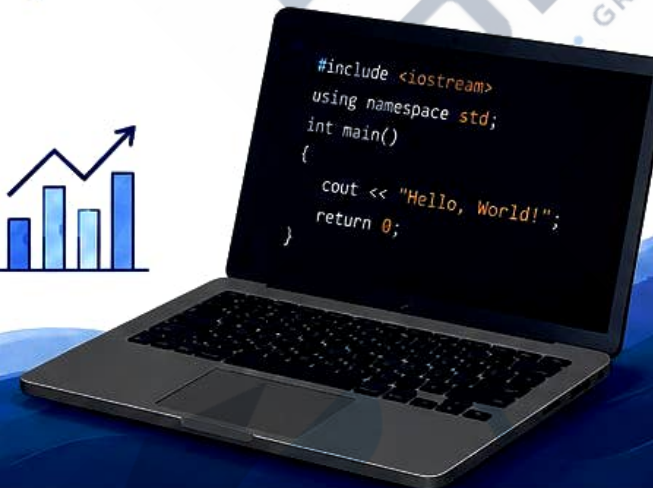
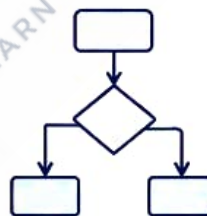
1010101010
0101010101
1010101010
0101010101



Unit - III



10110
01001
10101



Introduction to Expert Systems

Unit - III
(112225)

1. What is an Expert System ?

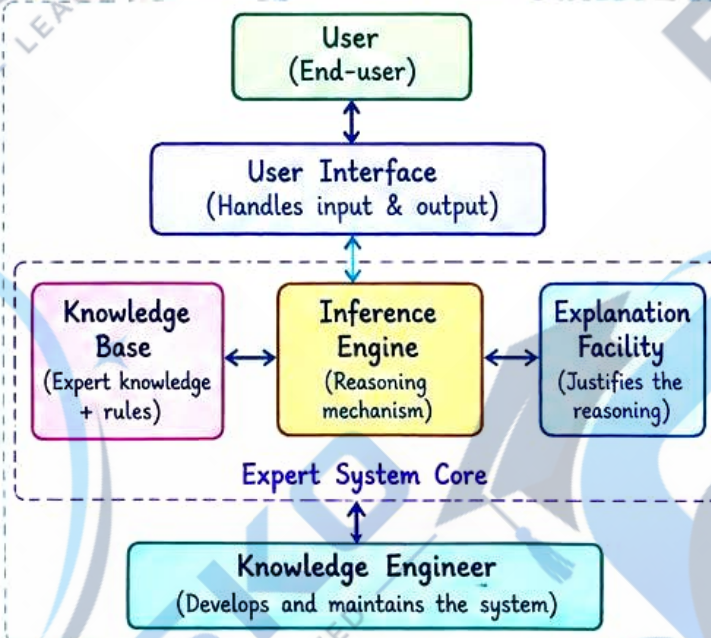
- An Expert System is a type of Artificial Intelligence system that uses knowledge and reasoning to solve problems in a specific domain, just like a human expert.
- It contains a knowledge base and an inference engine, which work together to provide expert-level advice or decisions.

"Expert systems capture the knowledge of human experts and make it available to a computer."

4. Types of Expert Systems

Type	Description
Rule-based Systems	Use IF-THEN rules for decision making. (most common)
Frame-based Systems	Use frames to represent knowledge (e.g., objects, attributes).
Model-based Systems	Use models to represent and solve problems (e.g., physical models).
Case-based Reasoning (CBR)	Solve new problems based on past similar cases.
Hybrid Systems	Combine two or more approaches (e.g., rule + case-based).

2. Structure of an Expert System



3. Components

- Knowledge Base**
 - Stores domain knowledge (facts, rules, heuristics).
 - Usually created by domain experts.
- Inference Engine**
 - Applies logical rules to the knowledge base.
 - Uses reasoning methods (forward or backward chaining).
- User Interface**
 - Allows user to interact with the system.
 - Takes input and displays results.
- Explanation Facility**
 - Explains how and why a particular conclusion was reached.
 - Builds trust and helps in debugging.
- Knowledge Engineer**
 - Works with domain experts to build and maintain the system.
 - Encodes knowledge in the system.

5. Working of an Expert System

- User enters a problem or query.
- Inference engine checks the knowledge base and applies rules.
- System finds the best solution / conclusion.
- Explanation facility shows the reasoning (if required).
- Result is presented to the user.

6. Example

Medical Diagnosis System

- Knowledge base: symptoms, diseases, rules.
- Inference engine: matches symptoms with rules.
- Example: If fever + cough + sore throat → may suggest "Common Cold".



7. Advantages & Limitations

Advantages

- ✓ Solves complex problems.
- ✓ Provides consistent results.
- ✓ Works 24x7 (no fatigue).
- ✓ Captures expert knowledge.
- ✓ Useful in many domains.

Limitations

- ✗ Knowledge acquisition is time consuming.
- ✗ Limited to specific domains.
- ✗ May give wrong results if knowledge is incomplete.
- ✗ Does not have human creativity or emotions.

Expert Systems = Human Expertise + Computer Power → Better Decisions → Smarter Solutions



Structure of Expert Systems

Unit - III
(112225)

1. Introduction

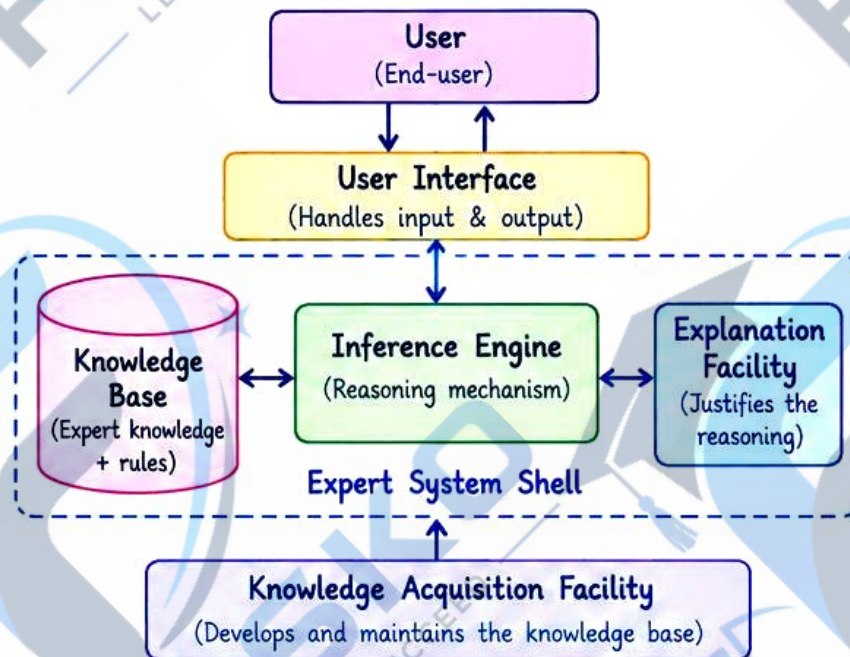
- An Expert System (ES) is a computer program that uses human expertise and knowledge to solve complex problems in a specific domain.
- It mimics the decision-making ability of a human expert.

"An expert system is a knowledge-based system that captures the expertise of a human and uses it to solve problems."

2. Main Components

- 1 Knowledge Base**
Stores domain knowledge (facts, rules, heuristics).
- 2 Inference Engine**
Applies logical rules to derive new facts and make decisions.
- 3 User Interface**
Allows user to interact with the system (asks questions, shows results).
- 4 Explanation Facility**
Provides reasoning steps and justifications for the conclusions.
- 5 Knowledge Acquisition Facility**
Helps in adding or updating knowledge from experts.

3. Structure / Architecture



4. Working of the Structure

- 1** User enters a problem or query through the user interface.
- 2** The inference engine checks the knowledge base and applies rules to find a solution.
- 3** The system may ask for more information (if needed) using the user interface.
- 4** The explanation facility provides the reasoning steps and justifies the result.
- 5** The result is displayed to the user.

5. Role of Each Component

- | | |
|---------------------------------------|---|
| Knowledge Base | → Contains domain facts, rules and heuristics. |
| Inference Engine | → Applies rules, performs reasoning and derives new facts. |
| User Interface | → Takes input from user and shows output. |
| Explanation Facility | → Provides "why" and "how" of the decision. |
| Knowledge Acquisition Facility | → Gets knowledge from experts and updates the knowledge base. |

6. Example

Medical Diagnosis Expert System

- Knowledge base: symptoms, diseases, rules.
- Inference engine: matches symptoms with rules.
- Explanation facility: explains why a disease is suggested.
- User interface: accepts patient details and shows diagnosis.



7. Key Points

- ✓ Uses knowledge and rules.
- ✓ Main components: KB, Inference Engine, UI, Explanation, Knowledge Acquisition.
- ✓ Helps in complex decision-making.
- ✓ Widely used in medical, finance, engineering, customer support, etc.

"Expert Systems = Human Expertise + Computer Power = Better Decisions"

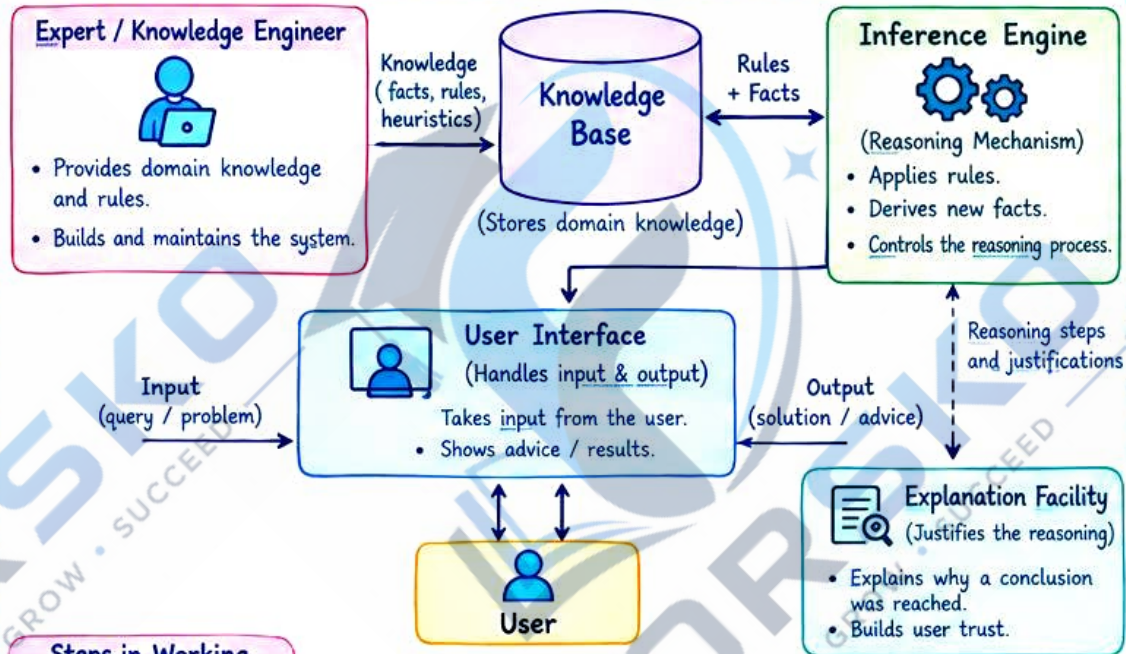


How Expert Systems Work and Problem Areas Addressed by Expert Systems

Unit - III
(112225)

1. How Expert Systems Work

An expert system works by capturing expert knowledge, reasoning with it, and providing advice or solutions to the user.



Steps in Working

- 1 Input** → User enters a problem or query through the interface.
- 2 Knowledge Retrieval** → The system fetches relevant knowledge from the knowledge base.
- 3 Reasoning** → The inference engine applies rules and logic to derive a solution.
- 4 Output** → The system provides advice, recommendation or solution.
- 5 Explanation** → The explanation facility shows the reasoning steps (if needed).

✓ In short → User + Knowledge Base + Inference Engine = Expert System

2. Problem Areas Addressed by Expert Systems

Expert systems are useful in areas where human expertise is complex, expensive, or difficult to access. They help in solving problems that require specialized knowledge, experience and logical decision-making.

1 Medical Diagnosis



- Helps in diagnosing diseases from symptoms.
- Assists in treatment recommendations.
- Example: MYCIN (blood infections).

2 Finance and Business



- Supports investment and financial planning.
- Detects fraud and manages risk.
- Example: Credit scoring, loan approval systems.

3 Agriculture



- Helps in crop disease detection.
- Recommends fertilizers and pesticides.
- Example: Plant disease diagnosis systems.

4 Engineering & Manufacturing



- Assists in design, troubleshooting and maintenance.
- Improves quality control and process optimization.
- Example: Fault diagnosis in machines, production planning.

5 Education & Training



- Provides personalized learning and tutoring.
- Assesses student performance.
- Example: Intelligent tutoring systems.

6 Customer Support / Help Desks



- Answers customer queries automatically.
- Provides product recommendations.
- Example: Chatbots with expert knowledge.



"Expert systems solve real-world problems by using specialized knowledge, logical reasoning and smart decision-making."



Types of Expert Systems

Unit - III
(112225)

Expert Systems can be classified into different types based on their purpose, knowledge representation, and the area of application. Each type is designed to handle specific kinds of problems and user needs.

1. Rule-Based Expert Systems



- Use a set of IF-THEN rules to represent knowledge.
- Reasoning is done through rule matching and inference.
- Simple, easy to build and maintain.

Example: MYCIN (medical diagnosis), XCON (computer configuration).

2. Case-Based Reasoning (CBR) Systems



- Solve new problems by referring to past cases (similar situations).
- Uses a case library and similarity matching.
- Learns and improves over time.

Example: puD (customer support), TAXON (financial planning).

3. Model-Based Expert Systems



- Use a model of the real world (e.g., physical, mathematical, or logical model).
- Reasoning is done using simulation or analysis.
- Useful for complex, dynamic systems.

Example: DENDRAL (chemical analysis), GENESIS (circuit design).

4. Fuzzy Expert Systems



- Handle uncertain, imprecise or vague information.
- Use fuzzy logic instead of strict true/false values.
- Suitable for real-world problems with partial data.

Example: Fuzzy control systems (e.g., washing machines, air conditioners).

5. Neural Network Expert Systems



- Use artificial neural networks to learn from data.
- Good at pattern recognition and classification.
- Improve performance with experience (learning).

Example: Diagnosis systems, image recognition, weather forecasting.

6. Hybrid Expert Systems



- Combine two or more types of expert systems (e.g., rule-based + neural network).
- Take advantage of multiple approaches for better accuracy.
- Handle complex and real-world problems more effectively.

Example: Intelligent Tutoring Systems, medical diagnosis (rule-based + neural network).

Key Takeaways



- ✓ Different types suit different problem domains.
- ✓ Choice of type depends on the nature of knowledge, data and application.
- ✓ Hybrid systems are often more powerful and flexible.

"Different problems,
different knowledge,
different expert systems."



Expert Systems and Internet / Web

Unit - III
(112225)



The Internet and Web provide a global platform, and together with expert systems, they enable intelligent, accessible and collaborative solutions.

1. What is the Connection?

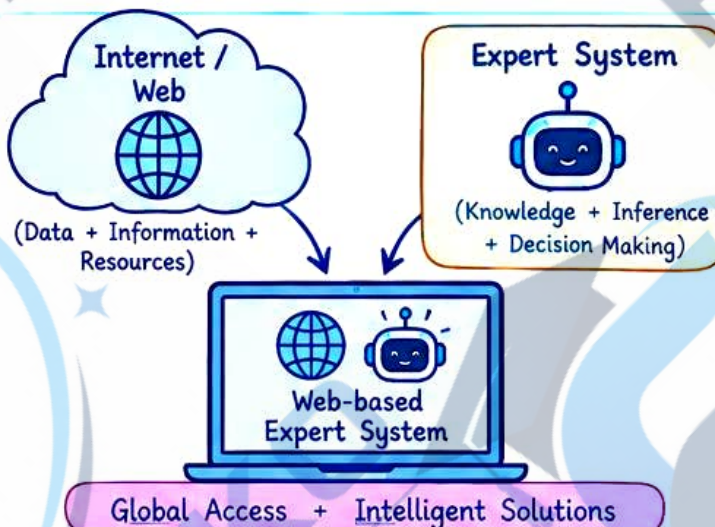
- The Internet / Web provides a vast source of data, knowledge and resources.
- Expert systems use this information to solve problems and provide intelligent advice.
- Together, they form web-based expert systems that can be accessed from anywhere in the world.

“The Internet gives expert systems a global reach.”

3. Role of the Internet / Web

- Access to large databases and information sources (e.g., medical, financial, scientific).
- Enables communication and collaboration between experts and users.
- Provides platforms like websites, web services, cloud and APIs.
- Helps in updating expert knowledge in real time.

Example: Online medical diagnosis systems, e-commerce recommendation systems, search engines with expert advice.



Role of Expert Systems

- Process and analyze the information received from the web.
- Use stored knowledge and rules to make decisions.
- Provide accurate, consistent and fast responses.
- Can be integrated into web applications, portals or cloud systems.

Example: Web-based health diagnosis, stock market analysis, online customer support (chatbots), travel planning systems.

How They Work Together

- 1 **Data Collection** – The web provides data (articles, databases, user inputs, etc.).
- 2 **Knowledge Processing** – Expert system analyzes the data using its knowledge base and inference engine.
- 3 **Decision / Advice** – It gives solutions, recommendations or expert advice.
- 4 **Delivery** – Results are shown through a web interface (browser / web app).

5. Applications

	Healthcare	Online disease diagnosis, health advice.
	Finance	Investment advice, fraud detection.
	Education	Personalized learning, doubt solving.
	E-commerce	Product recommendation, customer support.
	Search Engines	Smart search with expert results.
	Agriculture	Crop disease detection, farming advice.
	Industrial	Process control, maintenance advice.

★ Key Benefits

- ✓ Global access to expert knowledge.
- ✓ Faster and better decision-making.
- ✓ 24×7 availability.
- ✓ Cost-effective and scalable.
- ✓ Improved accuracy and reliability.
- ✓ Helps bridge the knowledge gap.

“The Internet connects the world, and expert systems make it smarter.”





Knowledge Engineering

The process of acquiring, structuring and representing knowledge so that it can be used by an expert system to solve problems.

Unit - III
(112225)

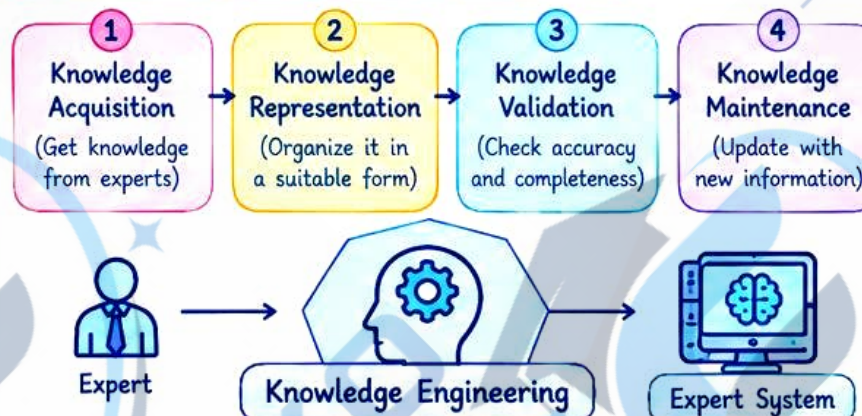
"From Human
Expertise to
Machine Intelligence"

1. What is Knowledge Engineering?

- It is the process of capturing knowledge from human experts and converting it into a form that can be used by an expert system.
- It involves techniques, tools and methodologies to represent, manage and maintain knowledge.
- It bridges the gap between human expertise and computer systems.

"Knowledge Engineering is the foundation of an intelligent system."

2. Knowledge Engineering Process



3. Key Activities

- Knowledge Elicitation**
Interact with experts using interviews, observation, questionnaires, etc.
- Knowledge Modeling**
Analyze and organize the knowledge using suitable structures.
- Knowledge Representation**
Use languages and models like rules, frames, ontologies, semantic networks, etc.
- Knowledge Verification & Validation**
Check for consistency, correctness and completeness.
- Knowledge Maintenance**
Update and refine knowledge over time.

4. Sources of Knowledge

- Human Experts**
Doctors, engineers, domain specialists.
- Documents**
Books, research papers, manuals.
- Databases**
Structured data and historical records.
- Web Resources**
Websites, online databases, forums.
- Experience**
Past cases and real-world situations.

5. Knowledge Representation Techniques

Technique	Description	Example
Production Rules	If-Then rules to represent expert knowledge.	IF fever = high THEN suspect infection.
Frames	Structured data with slots and values.	Frame: Patient Slots: name, age, symptoms...
Semantic Networks	Graph-based representation of concepts and relations.	(Disease) → causes → (Fever)
Ontologies	Define classes, relationships and hierarchy.	Animal → Mammal → Dog
Fuzzy Logic	Handles uncertain and imprecise knowledge.	Temperature is HIGH (0.7)

6. Tools and Technologies

- Knowledge Representation Languages**
(e.g., CLIPS, Prolog, OWL)
- Knowledge Acquisition Tools**
(e.g., Protégé, Expert Sourcing Tools)
- Knowledge Management Systems**
(e.g., Ontology editors, Semantic web tools)
- Machine Learning (in modern KE)**
(helps in automatic knowledge extraction)
- Expert System Development Environments**
(e.g., JESS, Drools)

Challenges

- Difficulty in acquiring knowledge from experts.
- Knowledge may be incomplete, inconsistent or uncertain.
- High cost and time consumption.
- Maintaining and updating knowledge is complex.

8. Applications

- Medical diagnosis
- Industrial troubleshooting
- Banking & finance
- Customer support (chatbots)
- Agriculture (crop advice)
- Education (tutoring systems)
- Cybersecurity (threat analysis)
- Legal systems (case analysis)

"Knowledge Engineering turns human expertise into reusable, reliable and intelligent knowledge."





Scope of Knowledge

Unit - III
(112225)

The scope of knowledge refers to the range, types and areas of knowledge that an expert system can acquire, represent and use to solve problems within a specific domain.

1. Types of Knowledge



Factual Knowledge

- Facts, data, events, objects.
- Example: "Paris is the capital of France."



Procedural Knowledge

- Steps, methods, algorithms.
- Example: "How to diagnose a disease."



Heuristic Knowledge

- Rules of thumb, experience-based.
- Example: "If the fever is high, check for infection."



Meta Knowledge

- Knowledge about the knowledge itself.
- Example: "Which rule to apply first?"

2. Scope in Different Domains



Medical

Disease diagnosis, treatment plans, drug interactions, patient data.



Finance

Investment analysis, risk assessment, credit scoring, fraud detection.



Agriculture

Crop advice, soil analysis, pest control, yield prediction.



Engineering

Design rules, fault diagnosis, maintenance, safety checks.



Education

Personalized learning, assessment, tutoring.



Business

Market analysis, decision support, customer service, planning.

3. Key Characteristics

- ✓ Domain-specific – covers a particular field.
- ✓ Wide but focused – includes relevant areas within the domain.
- ✓ Structured and organized – stored in a knowledge base.
- ✓ Can be updated – grows with new information.
- ✓ May include both explicit and implicit knowledge (from experts and experience).

4. Factors Affecting the Scope

- ★ Domain complexity – simple or complex domain.
- ★ Expert availability – quality of domain experts.
- ★ Knowledge acquisition methods – interviews, documents, observation, data mining.
- ★ System purpose – narrow or broad goals.
- ★ Available resources – time, cost, technology.

5. Importance



- Determines the system's ability to solve real-world problems.
- Affects accuracy and reliability of the system.
- Helps in selecting the right knowledge acquisition techniques.
- Essential for expanding the system to new areas or applications.

"The wider and deeper the knowledge,
the smarter the expert system."



Difficulties in Knowledge Acquisition

Unit - III
(112225)

Knowledge acquisition is the process of obtaining knowledge from experts, experiences or other sources and representing it in a form that can be used in an expert system.

Difficulties in Knowledge Acquisition :

The process of knowledge acquisition is not easy. It faces many challenges and difficulties, such as :

1. **Tacit Knowledge** → Experts often have implicit knowledge (experience, intuition, judgment) which is hard to express in words.
2. **Knowledge Complexity** → Real-world problems involve complex, uncertain and interconnected knowledge, making it difficult to capture accurately.
3. **Inconsistency** → Different experts may have different opinions or ways of solving the same problem.
4. **Limited Availability of Experts** → Domain experts are rare, busy and expensive. It is not always possible to get their time.
5. **Difficulty in Elicitation** → Experts may not be able to articulate their knowledge clearly, even when they are willing to help.
6. **Changing Knowledge** → Knowledge in many domains is constantly evolving. It is difficult to keep the system up to date.
7. **Knowledge Representation Issues** → Choosing the right representation scheme (rules, frames, etc.) is challenging and may lead to loss of information.
8. **Verification and Validation** → It is hard to check whether the acquired knowledge is correct, complete and consistent.
9. **Cost and Time** → Knowledge acquisition can be time-consuming and expensive, especially for large and complex domains.
10. **Human Factors** → Experts may be reluctant to share knowledge due to fear of job loss, privacy or lack of interest.

→ Despite these difficulties, effective knowledge acquisition techniques (interviews, observation, questionnaires, document analysis, etc.) are essential for building successful expert systems.

Methods of Knowledge Acquisition

Unit - III
(112225)

1. Expert Interview

- Involves direct conversation with domain experts (human specialists).
- The expert shares their knowledge, experience and problem-solving approach.
- Usually conducted in a structured or semi-structured way.



Best for: complex and specialized domains (e.g., medicine, law).

2. Questionnaires

- A set of well-structured questions is prepared and given to experts or knowledgeable people.
- Can be open-ended or multiple-choice.
- Useful when multiple experts are needed.



Best for: collecting knowledge from many people in a short time.

3. Observation

- Involves watching experts perform their tasks in real situations.
- Helps in understanding actual work practices and procedures.
- Useful for capturing tacit knowledge (knowledge not easily written).



Best for: practical and real-time knowledge (e.g., medical procedures, factory work).

4. Document Analysis

- Knowledge is extracted from existing documents, such as manuals, reports, books, research papers, and databases.
- Helps in getting accurate and well-documented information.



Best for: well-defined and stable domains (e.g., technical manuals, policies).

5. Brainstorming

- A group of experts discuss and share ideas freely.
- Helps in generating new ideas and uncovering hidden knowledge.
- Often used in combination with other methods.



Best for: creative and innovative domains (e.g., design, planning).

6. Machine Learning / Data Mining

- Extracts knowledge automatically from large amounts of data.
- Uses algorithms to find patterns, trends and relationships.
- Complements human-based methods.



Best for: large datasets and dynamic environments (e.g., market analysis, user behavior).



Key Takeaway

No single method is sufficient. In practice, multiple methods are combined to acquire accurate, complete and reliable knowledge for an expert system.

Machine Learning

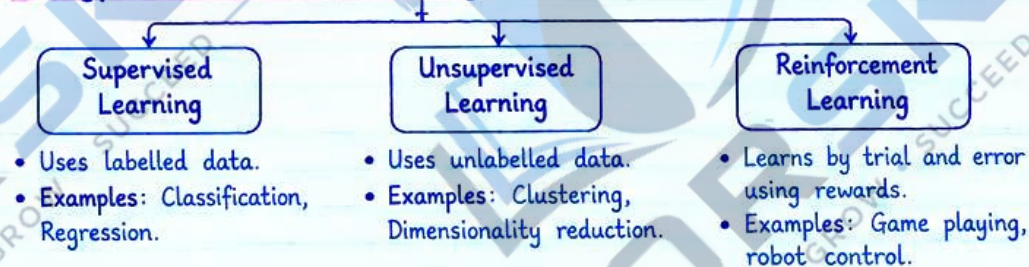
1. Introduction

- Machine Learning (ML) is a branch of Artificial Intelligence (AI) that allows computers to learn from data and improve their performance without being explicitly programmed.
- It focuses on building algorithms that can find patterns in data and make decisions or predictions.
- ML is widely used in areas like healthcare, finance, recommendation systems, image recognition, etc.

2. Key Elements

- Data** : Raw information used for training.
- Model** : Learns patterns from data.
- Algorithm** : Finds the best way to learn from data.
- Training** : Process of adjusting the model.
- Prediction** : Uses the trained model to make decisions.

3. Types of Machine Learning



4. Applications

- Speech and image recognition
- Email spam detection
- Self-driving cars
- Recommendation systems (e.g. Amazon, Netflix)

5. Advantages

- Improves over time.
- Handles large amounts of data.
- Solves complex problems.
- Automates decision making.

6. Conclusion

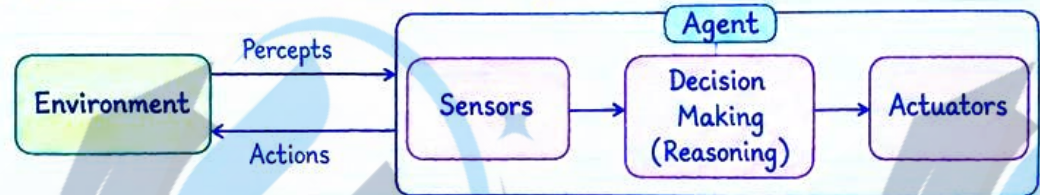
Machine Learning helps computers learn from data and make smart decisions, making our lives easier and better.

Intelligent Agents

1. Introduction

- An Intelligent Agent is a system that perceives its environment through sensors and acts on it through actuators to achieve a goal.
- It can be a software program or a physical robot.
- Intelligent agents are the building blocks of Artificial Intelligence.

2. Structure of an Intelligent Agent



3. Behaviour of Intelligent Agents

- Perceive** : Takes input from the environment using sensors.
- Think** : Processes the information and decides what to do.
- Act** : Performs actions using actuators.
- Learn** : Improves performance over time (in some agents).

4. Types of Agents (based on behaviour)

- Simple Reflex Agent** : Acts only on current percepts.
- Model-Based Reflex Agent** : Uses internal state (memory).
- Goal-Based Agent** : Tries to achieve a goal.
- Utility-Based Agent** : Chooses the best action based on utility.
- Learning Agent** : Learns from experience to improve.

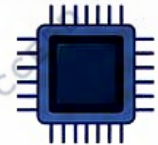
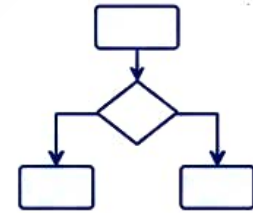
5. Examples

- Virtual assistant (e.g., Siri, Alexa)
- Chatbots
- Self-driving cars
- Robots
- Recommendation systems
- Smart home devices

6. Conclusion

Intelligent agents sense, think and act to achieve their goals, making them useful in many real-world applications.

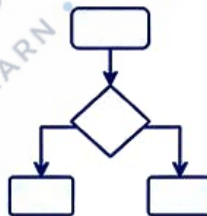
1010101010
0101010101
1010101010
0101010101



Unit - IV



10110
01001
10101



"Natural Language is the language of the real world."

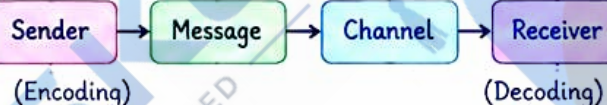
Understanding Natural Languages

Unit - IV
Artificial Intelligence

1. Introduction

- Natural Language (NL) is the language used by humans for everyday communication.
- It is informal, flexible and context-dependent.
- Understanding natural language is an important area of AI called Natural Language Processing (NLP).
- It helps computers to understand, interpret and generate human language.
- Examples: English, Hindi, Marathi, etc.

2. Components and Steps of Communication



Main Components:

- Sender** - the person who sends the message
- Message** - the information to be communicated
- Channel** - medium of communication (speech, text, etc.)
- Receiver** - the person who receives the message
- Context** - situation in which communication takes place
- Feedback** - response from receiver

Steps of Communication:

- Idea formation (what to say)
- Encoding (convert idea into language)
- Transmission (send through channel)
- Decoding (interpret the message)
- Understanding and feedback

Successful communication requires a common language, shared context and correct interpretation.



3. Formal vs Natural Languages

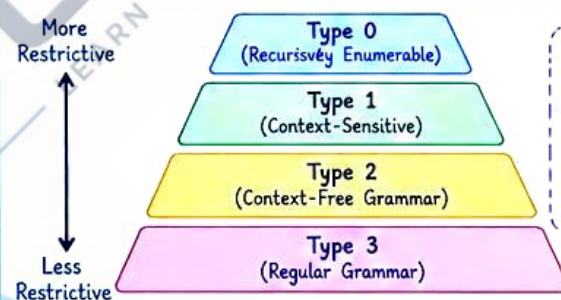
Aspect	Formal Language	Natural Language
Definition	A precisely defined language with strict syntax and rules.	A human language used in daily communication.
Examples	Mathematical notation, Programming languages	English, Hindi, Marathi, etc.
Syntax	Strict and unambiguous	Flexible and often ambiguous
Meaning	Exact and well-defined	Context-dependent
Use	Used in computers, logic and mathematics	Used by humans for communication
Ambiguity	No ambiguity	Often ambiguous
Learning	Easier for machines	Difficult for machines

4. Grammar and its Role

- Grammar is a set of rules that defines how sentences are formed in a language.
- It specifies the structure and order of symbols (words).
- Grammar helps in generating valid sentences and in analyzing given sentences.
- In NLP, grammar is used for parsing and understanding the structure of sentences.

Grammar defines the valid sentences of a language.

5. Chomsky Hierarchy of Grammars



Proposed by Noam Chomsky to classify formal grammars based on their generative power.

6. Parsing and Semantics

- Parsing:** The process of analyzing a sentence to find its grammatical structure (syntactic structure).
- It produces a parse tree showing how the sentence is formed.
- Semantics:** Deals with the meaning of sentences.
- After parsing, semantic analysis checks whether the sentence makes sense in a given context.
- Example:



7. Parsing Techniques

- Top-Down Parsing:** Starts from the start symbol and tries to generate the input string. (e.g., Recursive Descent, Predictive Parsing)
- Bottom-Up Parsing:** Starts from the input string and tries to reach the start symbol. (e.g., Shift-Reduce Parsing, LR Parsing)
- Parsing techniques are used to build a parse tree for a given sentence.

8. Context-Free Grammars (CFG)

- A Context-Free Grammar is a grammar in which each production rule has a single non-terminal symbol on the left-hand side.
- Formally, a CFG is a 4-tuple: $G = (V, T, P, S)$ where,
 - V = set of variables (non-terminals)
 - T = set of terminals
 - P = set of production rules
 - S = start symbol

- Example:

$S \rightarrow NP VP$
 $NP \rightarrow Det N \mid N$
 $VP \rightarrow V NP$
 $Det \rightarrow the \mid a$
 $N \rightarrow boy \mid ball$
 $V \rightarrow kicked \mid saw$

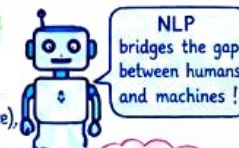
CFG is used to represent the syntax of natural languages.

9. Transformational Grammars

- Proposed by Noam Chomsky.
- Extends context-free grammars by adding transformational rules.
- These rules convert a basic sentence structure into various forms.
- Example: Active $\rightarrow$ Passive, Statement $\rightarrow$ Question, etc.

10. Recursive Transition Nets (RTN)

- A graphical notation for representing grammar.
- Uses finite state machines with recursive procedures.
- Each network represents a grammatical construct (phrase/sentence).
- Useful in natural language understanding systems.



From Words the Understanding ...

Natural Language Understanding = Syntax (Structure) + Semantics (Meaning) + Context (Real World)

Components and Steps of Communication

"Communication is the bridge between minds."

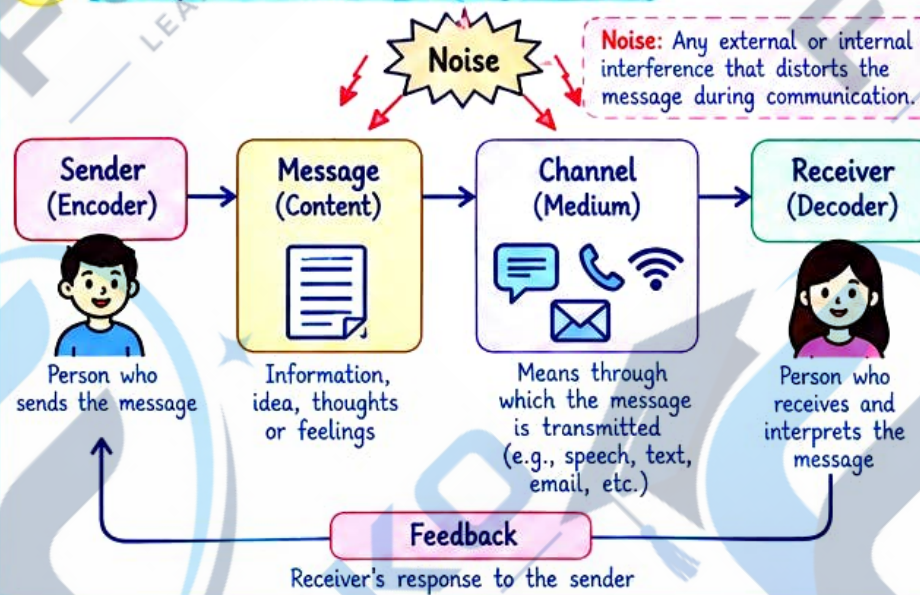
1. Introduction

- Communication is the process of sharing information, ideas, thoughts, or feelings from one person (or system) to another.
- In the context of Natural Language Processing (NLP), communication usually refers to human-to-human communication using natural language (e.g., English, Hindi, etc.).
- It involves encoding a message, sending it through a medium, and decoding it to understand the intended meaning.

4. Steps of Communication

- Idea Formation**
Sender has an idea or thought to communicate.
- Encoding**
The idea is converted into a message using language (words, symbols, gestures, etc.).
- Transmission**
The message is sent through a channel or medium.
- Decoding**
Receiver interprets the message and tries to understand its meaning.
- Feedback**
Receiver responds to the sender.

2. Components of Communication



5. Example of Communication



6. Importance in NLP

- ✓ Helps in understanding how humans use language.
- ✓ Essential for designing systems like chatbots, speech recognition, machine translation, etc.
- ✓ Consideration of noise, context, and feedback improves communication quality.
- ✓ Forms the basis for Natural Language Processing and understanding human-computer interaction.

3. Description of Components

- Sender (Encoder)**
 - Person who initiates the communication.
 - Converts idea into a message (encoding).
- Message**
 - The actual information, idea, or content being communicated.
 - Can be in the form of words, gestures, images, etc.
- Channel (Medium)**
 - The means through which the message is sent.
 - Examples: speech, text, phone, email, internet, etc.
- Receiver (Decoder)**
 - Person who receives the message.
 - Decodes and interprets the message.
- Feedback**
 - Receiver's response to the sender.
 - Helps to confirm whether the message is understood.
- Noise**
 - Any interference that distorts the message.
 - Types: physical noise (loud sound), semantic noise (wrong meaning), psychological noise (distraction).

7. Key Takeaways

- Communication is a two-way process.
- It involves sender, message, channel, receiver, feedback, and noise.
- Each component plays an important role.
- Effective communication happens when the message is correctly understood with minimal noise.
- These principles are fundamental for building intelligent systems that can understand and generate human language.



"Good communication leads to better understanding."

Clear Communication = Correct Encoding + Effective Channel + Proper Decoding + Useful Feedback - Minimal Noise

The Contrast between Formal and Natural Languages in the Context of Grammar

"Formal languages have precise rules, Natural languages have human touch."

1. Introduction

- Language is a system of symbols used for communication.
- In AI and NLP, we study both formal languages and natural languages.
- Grammar defines the set of rules used to generate valid sentences in a language.
- Formal and natural languages differ in their purpose, structure, ambiguity and complexity.



2. What is a Formal Language?

- A formal language is a set of strings over an alphabet, defined by precise mathematical rules (grammar).
- It is used in theoretical computer science and programming languages.
- It is artificial, designed by humans for specific purposes.

Example: Arithmetic expression language:
 $L = \{ a + b \mid a, b \text{ are numbers} \}$

3. What is a Natural Language?

- A natural language is a language that evolves naturally among humans for everyday communication.
- It is rich, flexible, context-dependent and often ambiguous.
- It follows grammatical rules, but these rules are not strict or complete.
- Examples: English, Hindi, Marathi, etc.

Example:

- English: "The cat is on the mat."
- Hindi: "बिल्ली चटाई पर है।"
- Marathi: "मोंजर चढाईवर आहे."



4. Comparison: Formal vs Natural Languages (in the context of Grammar)

Aspect	Formal Language	Natural Language
Origin	Artificial, designed by humans	Evolves naturally among humans
Purpose	Used for computation, programming, and theoretical study	Used for everyday human communication
Grammar	Strict, well-defined, mathematical rules (e.g., CFG, regular grammar)	Rules exist, but are flexible and often incompletely defined
Syntax	Precise and unambiguous	Flexible and can be ambiguous
Semantics	Well-defined, single meaning	Context-dependent, multiple meanings
Ambiguity	Generally no ambiguity	Often ambiguous (lexical, syntactic, semantic)
Complexity	Relatively simple and limited	Extremely complex
Examples	Programming languages (C, Java), mathematical languages	English, Hindi, Marathi, etc.

5. Examples

Formal Language Example

Grammar:

$E \rightarrow E + E \mid E * E \mid (E) \mid id$

Valid strings:

- $a + b$
- $(a + b) * c$
- $id * id + id$

Here, the rules are precise and only correctly formed expressions are allowed.

Natural Language Example

- Sentence: "I saw the girl with the telescope."
- This sentence is grammatically correct but ambiguous:
 - I used a telescope to see the girl.
 - I saw a girl who had a telescope.

Natural languages allow multiple interpretations, and meaning depends on context.

6. Key Differences in Grammar

Formal Grammar

- Defined using formal rules (e.g., regular grammar, context-free grammar).
- Generates a well-defined set of strings.
- Used in automata theory and compilers.

VS

Natural Grammar

- Based on human usage and conventions.
- Allows exceptions, informal expressions, and context-based meanings.
- Harder to model formally.
- Studied in linguistics and NLP.

7. Importance in AI and NLP

- Formal languages help in designing parsers, compilers, and language models.
- Understanding natural language allows computers to interact with humans.
- Bridging the gap between formal and natural languages is a key challenge in NLP.

8. Key Takeaways

- ✓ Formal languages are precise and unambiguous.
- ✓ Natural languages are flexible and context-dependent.
- ✓ Grammar in formal languages is strict; in natural languages it is more flexible.
- ✓ Understanding both is essential for building intelligent systems that can process human language.

Formal Languages give machines exact instructions + Natural Languages give humans endless expressions → Together they build Intelligent Communication.

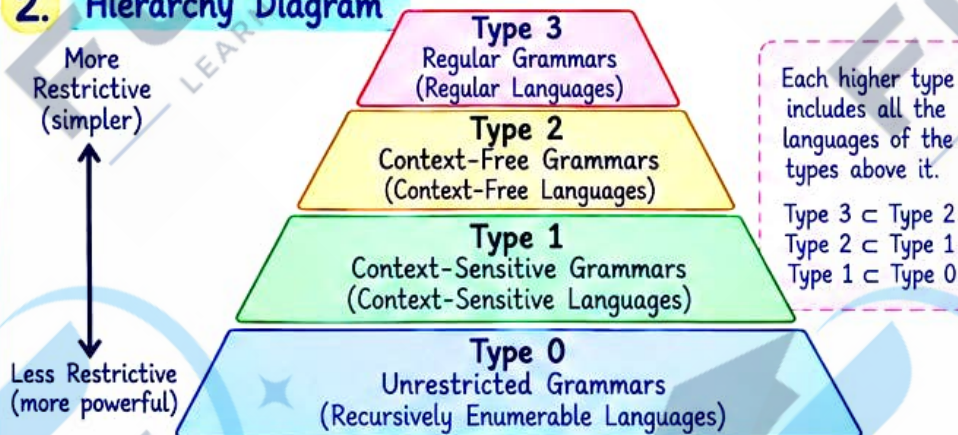
Chomsky Hierarchy of Grammars

"A hierarchy that classifies grammars based on their generative power."

1. Introduction

- The Chomsky hierarchy classifies formal grammars into four types based on the form of production rules and the type of languages they generate.
- Proposed by Noam Chomsky in 1956.
- It helps in understanding the power and limitations of different grammatical models and the corresponding automata.
- It is widely used in compilers, NLP and formal language theory.

2. Hierarchy Diagram



3. Key Points

- Type 3 grammars are the simplest and generate regular languages.
- Type 2 grammars generate context-free languages and are widely used in programming language syntax.
- Type 1 grammars generate context-sensitive languages and are more powerful.
- Type 0 grammars are the most powerful and can generate any recursively enumerable language.
- Each type of grammar is recognized by a corresponding automaton model.
- The hierarchy shows increasing generative power (from Type 3 to Type 0).

4. Chomsky Hierarchy Table

Type	Grammar Name	Production Rule Form	Language Generated	Automaton Model	Example Language
Type 3	Regular Grammar	$A \rightarrow aB \mid A \rightarrow a$ (A, B are non-terminals, a is terminal)	Regular Languages	Finite Automaton (FA)	$L = \{a^n b^n \mid n \geq 0\}$ (e.g., strings of a's)
Type 2	Context-Free Grammar (CFG)	$A \rightarrow \alpha$ (A is non-terminal, α is string of terminals and/or non-terminals)	Context-Free Languages	Pushdown Automaton (PDA)	$L = \{a^n b^n \mid n \geq 0\}$ (e.g., balanced parentheses)
Type 1	Context-Sensitive Grammar (CSG)	$\alpha A \beta \rightarrow \alpha \gamma \beta$ (α, β, γ are strings, $ \gamma \geq 1$)	Context-Sensitive Languages	Linear Bounded Automaton (LBA)	$L = \{a^n b^n c^n \mid n \geq 0\}$
Type 0	Unrestricted Grammar	$\alpha \rightarrow \beta$ (α, β are strings of terminals and/or non-terminals, $\alpha \neq \epsilon$)	Recursively Enumerable Languages	Turing Machine (TM)	$L = \{ \langle M, w \rangle \mid M \text{ halts on } w \}$

5. Example Productions

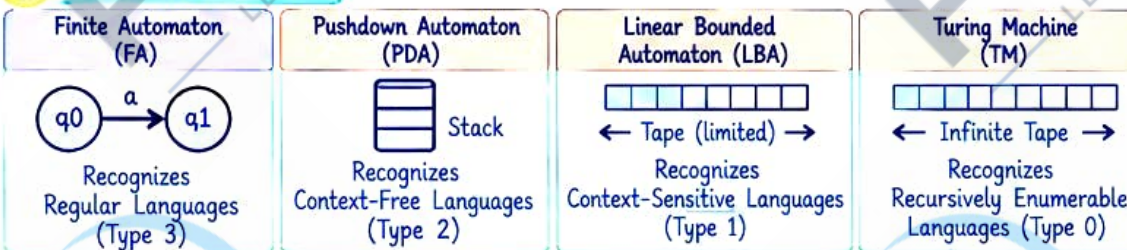
Type 3: $S \rightarrow aS \mid b \mid \epsilon$

Type 2: $S \rightarrow aSb \mid \epsilon$

Type 1: $\alpha A \beta \rightarrow \alpha \gamma \beta$ (e.g., $A \rightarrow aA$)

Type 0: $\alpha \rightarrow \beta$ (e.g., $S \rightarrow aSb \mid ab$)

6. Automata Models



7. Key Takeaways

- Chomsky hierarchy classifies grammars into four types based on production rules.
- Each higher type is more powerful.
- Each type corresponds to a specific automaton.
- It provides a theoretical foundation for compilers, programming languages and NLP.
- Helps to understand what can be computed and recognized.

Type 3 $\subset$ Type 2 $\subset$ Type 1 $\subset$ Type 0 $\rightarrow$ A hierarchy of increasing generative power

Parsing and Semantics

"Parsing gives structure to sentences, Semantics gives meaning."

1. Introduction

- Parsing and semantics are key processes in Natural Language Processing (NLP).
- Parsing analyzes the grammatical structure of a sentence.
- Semantics deals with the meaning of the sentence.
- Both work together to help computers understand human language.

2. What is Parsing?

- Parsing is the process of analyzing a sentence and determining its grammatical structure according to a grammar (e.g., Context-Free Grammar).
- It checks whether a sentence is syntactically correct and builds a parse tree.
- Parsing is based on the rules of grammar (production rules).

"Parsing tells us how the words are organized in a sentence."

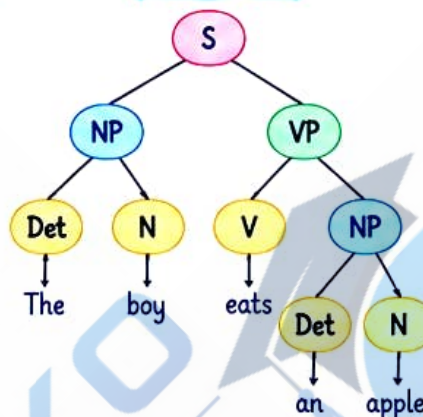
3. Example of Parsing

Sentence: "The boy eats an apple."

Grammar Rules (CFG)

$S \rightarrow NP VP$
 $NP \rightarrow Det N$
 $VP \rightarrow V NP$
 $Det \rightarrow the | an$
 $N \rightarrow boy | apple$
 $V \rightarrow eats$

Parse Tree



The parse tree shows the hierarchical structure of the sentence.

4. Types of Parsing

Type	Description	Example / Use
Top-Down Parsing	Starts from the start symbol and tries to generate the input string.	Recursive Descent Parsing, Predictive Parsing
Bottom-Up Parsing	Starts from the input string and tries to construct the start symbol.	Shift-Reduce Parsing, LR Parsing
Syntax Tree (Parse Tree)	Tree representation of the grammatical structure.	Shows syntactic structure of sentence.

5. What is Semantics?

- Semantics is the study of meaning in natural language.
- It analyzes the meaning of a sentence based on its syntactic structure and context.
- It helps in determining what the sentence actually conveys in the real world.
- Semantic analysis uses techniques like:
 - Semantic representation (e.g., logical form)
 - Predicate logic
 - Meaning rules

"Semantics tells us what the sentence means."

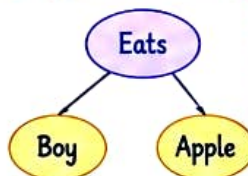
6. Example of Semantics

Sentence: "The boy eats an apple."

Meaning Representation (in Predicate Logic)

Boy(b)
 Apple(a)
 Eats(b, a)

Semantic Tree (Conceptual View)



This means "there exists a boy and an apple such that the boy eats the apple."

7. Parsing vs Semantics

Aspect	Parsing (Syntax)	Semantics (Meaning)
Purpose	Finds grammatical structure	Finds meaning
Output	Parse tree	Meaning representation (logical form)
Basis	Grammar rules (CFG)	Semantic rules, context, world knowledge
Checks	Syntactic correctness	Semantic correctness
Example	Tree structure of sentence	Predicate logic form

8. Importance

- Parsing helps in syntactic analysis and is necessary for semantic analysis.
- Semantics allows deeper understanding of language.
- Both are essential for applications like:
 - Machine translation
 - Chatbots and virtual assistants
 - Information extraction
 - Question answering systems
 - Natural language understanding (NLU)

Syntax gives form, Semantics gives meaning!



Parsing + Semantics = Correct Structure + Correct Meaning → Better Understanding of Human Language

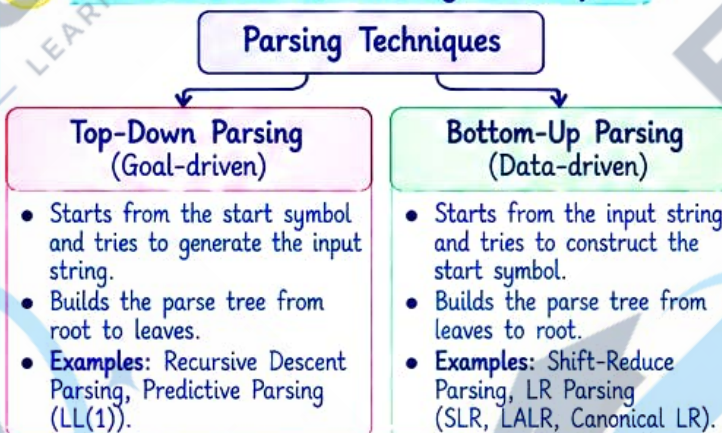
Parsing Techniques

"Parsing techniques analyze the syntactic structure of a sentence using a grammar."

1. Introduction

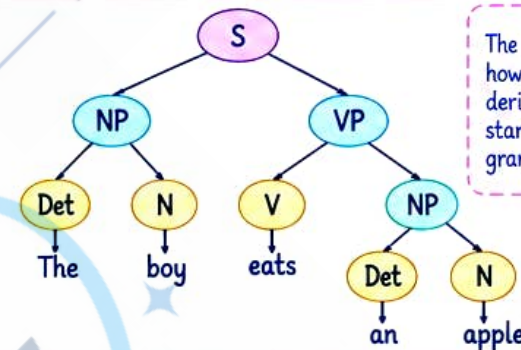
- Parsing is the process of analyzing a string of words (sentence) and determining its grammatical structure according to a grammar (usually CFG).
- Parsing techniques are systematic methods used to construct a parse tree for a given input sentence.
- They check whether the sentence is syntactically correct and show how it can be derived from the grammar rules.
- Parsing techniques are mainly classified into Top-Down and Bottom-Up parsing.

2. Classification of Parsing Techniques



3. Parse Tree (Example)

Sentence: "The boy eats an apple."



The parse tree shows how the sentence is derived from the start symbol S using grammar rules.

4. Top-Down Parsing Techniques

Technique	Description	Features
Recursive Descent Parsing	Uses recursive procedures for each non-terminal in the grammar.	- Simple to implement - Backtracking may be needed - Not efficient for left recursive grammars
Predictive Parsing (LL(1))	Uses a parsing table to predict the production rule (no backtracking).	- More efficient - Requires grammar to be left-factored and free from left recursion - Uses FIRST and FOLLOW sets

5. Example: Predictive Parsing (Top-Down)

Grammar:	Parsing Table (LL(1))					
	Non-terminal	the	an	boy	apple	eats
1. $S \rightarrow NP VP$	S	1	1	-	-	-
2. $NP \rightarrow Det N$	NP	2	2	-	-	-
3. $VP \rightarrow V NP$	VP	-	-	-	-	3
4. $Det \rightarrow the an$	Det	4	4	-	-	-
5. $N \rightarrow boy apple$	N	-	-	5	5	-
6. $V \rightarrow eats$	V	-	-	-	-	6

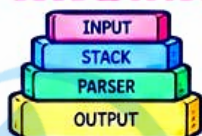
6. Bottom-Up Parsing Techniques

Technique	Description	Features
Shift-Reduce Parsing	Uses a stack to shift input symbols and reduce them using grammar rules.	- Builds parse tree from leaves to root - More efficient than top-down - Basis for LR parsing
LR Parsing (SLR, LALR, Canonical LR)	Advanced bottom-up parsing technique using action and goto tables.	- Handles a large class of grammars - No backtracking - Used in compilers (e.g., YACC, Bison)

7. Example: Shift-Reduce Parsing (Bottom-Up)

Stack	Input	Action
\$	the boy eats \$	Shift
\$ the	boy eats \$	Reduce (Det $\rightarrow$ the)
\$ Det	boy eats \$	Shift
\$ Det boy	eats \$	Reduce (N $\rightarrow$ boy)
\$ NP	eats \$	Shift
\$ NP eats	\$	Reduce (V $\rightarrow$ eats)
\$ NP VP	\$	Reduce (S $\rightarrow$ NP VP)
\$ S	\$	Accept

The parser shifts input symbols onto the stack and reduces them using grammar rules until the start symbol is obtained.



8. Comparison: Top-Down vs Bottom-Up

Aspect	Top-Down Parsing	Bottom-Up Parsing
Direction	Root to leaves	Leaves to root
Starting point	Start symbol	Input string
Approach	Goal-driven	Data-driven
Backtracking	May be required (except LL(1))	Not required
Grammar requirement	Cannot handle left recursion	Can handle left recursion
Examples	Recursive Descent, Predictive (LL(1))	Shift-Reduce, LR, SLR, LALR

9. Important Points

- ✓ Parsing techniques are essential in compilers and NLP.
- ✓ Top-Down parsing is simple but may require backtracking.
- ✓ Predictive parsing uses FIRST and FOLLOW sets.
- ✓ Bottom-Up parsing is more powerful and efficient.
- ✓ LR parsing is widely used in practical applications.
- ✓ The choice of technique depends on the grammar and application requirements.



"A good parser reveals the structure behind the sentence."

10. Key Takeaways

- Parsing determines the syntactic structure of a sentence.
- Top-Down and Bottom-Up are the two main approaches.
- Different parsing techniques are used based on efficiency and grammar constraints.
- Parsing is a fundamental step in understanding natural languages.

Parsing Techniques help convert a sequence of words into a structured form (parse tree) using grammar rules.

Context-Free and Transformational Grammars

"Grammar is the key that unlocks the structure and creativity of language."

1. Introduction

- Formal grammars are used to generate sentences in a language.
- Context-Free Grammars (CFG) are widely used in NLP for syntactic analysis (parsing).
- Transformational Grammars extend CFG with transformation rules to capture the deep and surface structure of sentences.
- Both are important for understanding the structure and generative nature of natural languages.

2. Context-Free Grammar (CFG)

Definition

A Context-Free Grammar (CFG) is a formal grammar in which each production rule has a single non-terminal on the left-hand side.

$$A \rightarrow \alpha$$

where A is a non-terminal and α is any string of terminals and/or non-terminals.

Formal Definition

A CFG is a 4-tuple:

$$G = (V, \Sigma, P, S)$$

where,

- V : finite set of non-terminals
- Σ : finite set of terminals
- P : set of production rules
- S : start symbol ($S \in V$)

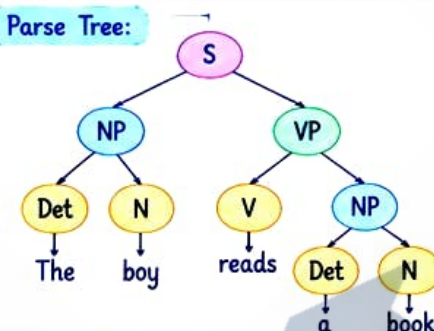
3. Example of CFG

Grammar rules:

- $S \rightarrow NP VP$
- $NP \rightarrow Det N$
- $VP \rightarrow V NP$
- $Det \rightarrow the | a$
- $N \rightarrow boy | girl | book$
- $V \rightarrow reads | writes$

Sentence: "The boy reads a book."

Parse Tree:



4. Properties of CFG

- Can generate nested structures (recursion).
- Well-suited for natural language syntax.
- Cannot directly handle context-sensitive dependencies (e.g., agreement, movement).
- Widely used in compilers and NLP parsers.
- Each rule has one non-terminal on the left side.
- The language generated by a CFG is called a Context-Free Language (CFL).

5. Derivation in CFG

Derive the sentence: "The boy reads a book."

$S \rightarrow NP VP$
 $\rightarrow Det N VP$
 $\rightarrow the N VP$
 $\rightarrow the boy VP$
 $\rightarrow the boy V NP$
 $\rightarrow the boy reads NP$
 $\rightarrow the boy reads Det N$
 $\rightarrow the boy reads a N$
 $\rightarrow the boy reads a book$

A parse tree or derivation shows how a sentence is generated from the start symbol using production rules.

6. Transformational Grammar

Definition

- Transformational Grammar (TG) was proposed by Noam Chomsky.
- It extends Context-Free Grammar by adding transformation rules.
- It consists of two levels:
 - Deep Structure:** underlying syntactic structure (basic meaning).
 - Surface Structure:** actual sentence form after transformations.
- Transformation rules convert deep structure into different surface structures.

7. Key Concepts in TG

1. Deep Structure

- Represents the core meaning.
- Abstract and underlying form.

2. Surface Structure

- Actual form of the sentence.
- What we speak or write.

3. Transformation Rules

- Rules that change deep structure to surface structure.
- Examples: move, insert, delete, substitute, etc.

4. Generative Power

- Explains how infinite sentences can be generated from finite rules.

8. Example of Transformational Grammar

Deep Structure:

The boy is in the room.

Surface Structures (after transformations):

- Is the boy in the room?
- The boy is in the room.
- The boy in the room is happy.
- In the room, the boy is.

Transformation Rule (Move):

Move auxiliary verb to the beginning (Question Formation).

The boy is in the room.



Is the boy in the room?

9. Comparison: CFG vs Transformational Grammar

Aspect	Context-Free Grammar (CFG)	Transformational Grammar (TG)
Introduced by	Used widely in formal language theory (early NLP)	Noam Chomsky
Main idea	Generation of sentences using production rules	Extends CFG with transformation rules
Structure	Single level (surface structure)	Two levels: deep and surface structure
Rules	$A \rightarrow \alpha$ (single non-terminal)	CFG rules + transformation rules
Captures meaning	Limited	Better (through deep structure)
Use	Parsing, compilers, basic NLP	Linguistic analysis, sentence variation

10. Key Takeaways

- ✓ CFG is a powerful model for syntactic structure and parsing.
- ✓ Transformational Grammar explains how sentences are transformed from deep to surface structure.
- ✓ TG helps in handling multiple sentence forms with the same meaning.
- ✓ Both are important for understanding human language and are widely used in NLP.

Context-Free Grammar gives the structure + Transformational Grammar gives the creativity → Together they model human language.

Recursive Transition Nets (RTNs)

"Recursive transition nets combine finite automata with procedures to handle natural language structure."

1. Introduction

- Recursive Transition Nets (RTNs) were proposed by Woods (1970) for natural language understanding.
- It is an extension of finite state machines with the ability to make recursive calls to subnets.
- RTNs can handle hierarchical and nested structures found in natural languages.
- It provides a graphical and procedural formalism for syntactic analysis (parsing).
- Suitable for context-free grammars and natural language parsing.

2. Basic Idea

- A transition network consists of states and arcs (transitions).
- Each arc is labeled with a word, a category, or a call to another subnet.
- If a subnet is called, the control goes to that subnet, and after it completes, control returns to the calling point (recursion).
- This allows the handling of nested structures like noun phrases, verb phrases, etc.

"RTNs = Finite State Machines + Recursive Calls to Subnets"

3. Components of an RTN

Component	Meaning
State (Node)	Represents a point in the network (usually a circle).
Transition (Arc)	Labeled with a word, category (e.g. N, V), or a subnet call.
Final State	Indicates successful completion (usually double circle).
Terminal Symbol	A specific word from the input sentence.
Non-terminal Symbol / Subnet	Represents a call to another transition network.
Return	After the subnet is successfully parsed, control returns to the next state in the calling network.

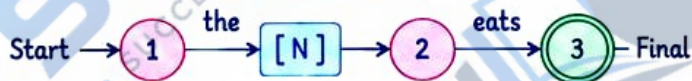
4. Notations Used

-  State (node)
-  Final state (accepting state)
-  Transition (arc)
-  Arc labeled with a word (e.g. 'the', 'boy')
-  Call to a subnet (non-terminal)
-  Return from subnet

A subnet is written in a separate network and called like a procedure.

5. Example: RTN for a Simple Sentence

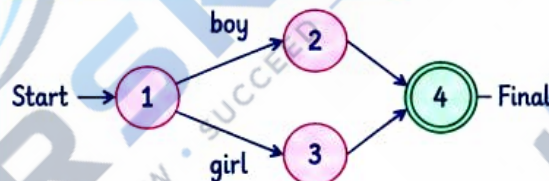
Sentence: "The boy eats."



Working:

1. Read 'the'.
2. Call subnet N (to recognize a noun).
3. Return and read 'eats'.
4. Reach final state (sentence accepted).

6. Subnet for Noun (N)



This subnet recognizes the nouns 'boy' or 'girl' and can be extended with more nouns.

7. Example with Recursion (Nested Structure)

Sentence: "The boy sees the girl."



The same N subnet is called twice (recursively) to recognize both noun phrases.

8. Advantages

- ✓ Can handle recursive and nested structures.
- ✓ Simple and intuitive graphical representation.
- ✓ Useful for natural language parsing.
- ✓ Equivalent in power to context-free grammars.
- ✓ Easy to implement for small to medium grammars.
- ✓ Modular design (subnets can be reused).

9. Limitations

- ✗ Not suitable for handling semantic analysis.
- ✗ Can become complex for large grammars.
- ✗ Does not handle context-sensitive features directly.
- ✗ Less powerful than modern parsing techniques (e.g. probabilistic parsers, deep learning models).
- ✗ Difficult to incorporate ambiguity resolution.

10. Applications

- Syntactic analysis (parsing) in Natural Language Processing (NLP).
- Building language understanding systems.
- Used in early NLP systems and chatbots.
- Can be used for structured input processing (e.g. commands).
- Educational tool to understand recursive grammar structures.

RTNs helped bridge formal grammar theory and practical NLP parsing.

Recursive Transition Nets provide a powerful and elegant way to parse natural languages by combining finite state machines with recursion.